Healthcare Clinic Management System

Royal Care Clinic — Object-Oriented Design and Python Implementation

Student name: [Your name]

Student ID: [Your ID]

Course: [Course code and title]

Repository: https://github.com/<your-username>/royal-care-clinic

Date: 28 October 2026

Contents

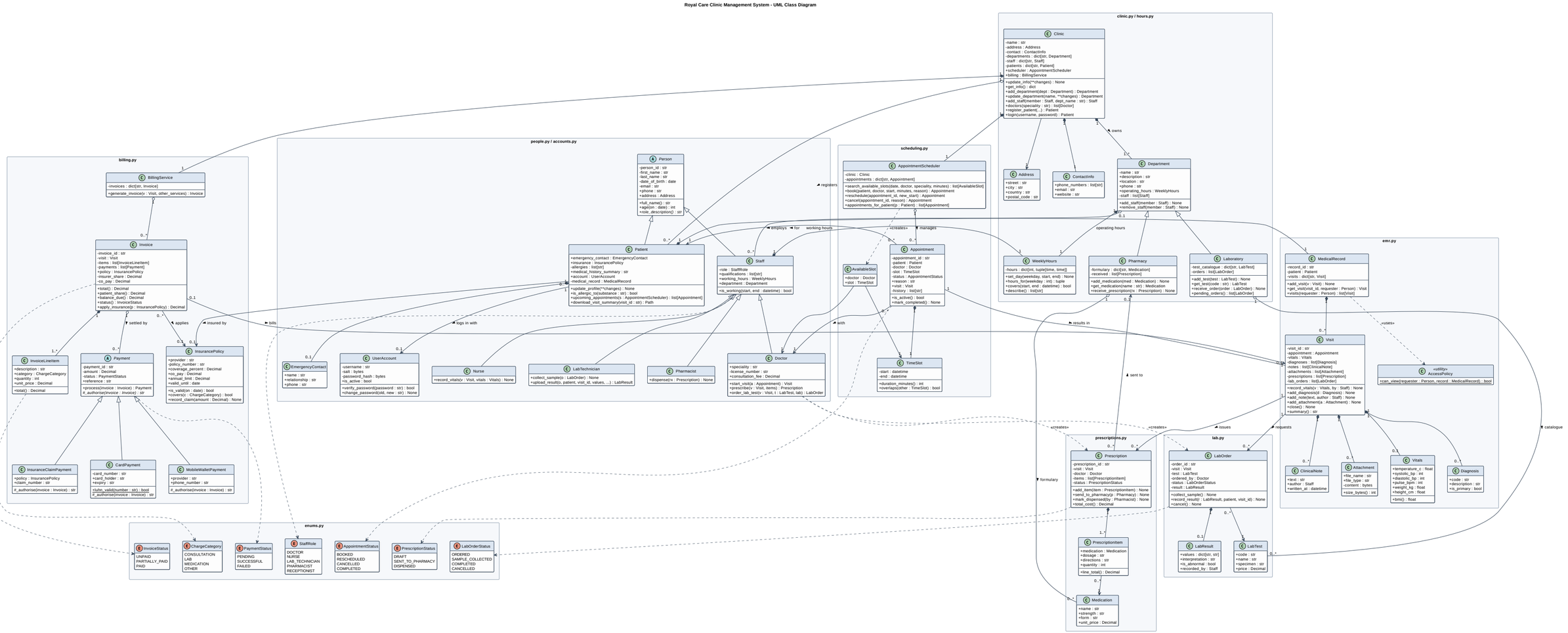
1. a. UML Class Diagram and Description
    1. Class diagram
    2. Classes and responsibilities
    3. Relationships
    4. Modularity
    5. Assumptions
    6. Design principles applied
2. b. Python Classes (source code), program output and test results
3. c. GitHub repository and commit history
4. d. Summary of learnings

Overview

The application manages an outpatient clinic. It covers clinic information and departments, patients and their accounts, doctors and staff, appointment scheduling, electronic medical records (EMR), prescriptions, laboratory orders and results, and billing with insurance co-pay and multiple payment methods. It is written in pure Python 3.10+ (standard library only) and organised as the package `royal_care`, with 15 modules. A demonstration program (`main.py`) and a separate test file (`tests/test_clinic_system.py`, 31 tests, all passing) are included.

Run: `python main.py` | Tests: `python -m unittest discover -s tests -v`

Figure 1. UML class diagram of the Royal Care Clinic Management System (PlantUML; packages correspond to Python modules). Vector image: zoom in to read details. The PNG/SVG files are also in the repository's docs/ folder.



a. UML Class Diagram and Description

a.1 Notation used

Symbol	Meaning
`- / + / #`	private / public / protected members (Python: `_name` with read-only `@property` = private; `_authorise` = protected hook)
«abstract», *italic* / {abstract}	abstract class / abstract method (Python `ABC` + `@abstractmethod`)
{static} (underlined)	static method (`@staticmethod`)
`A <|-- B`	Inheritance: B is a kind of A
`A *-- B`	Composition: B is part of A and is created and destroyed with it
`A o-- B`	Aggregation: A groups Bs, but B exists independently
`A --> B`	Directed association with multiplicities (1, 0..1, 0..*, 1..*)
`A ..> B`	Dependency: A uses or creates B temporarily

a.2 Classes and responsibilities

The design has 40 classes, enumerations included, well above the minimum of six.

Module	Class	Responsibility
clinic.py	**Clinic**	Clinic profile (name, address, contacts) and entry point to every service: departments, staff, patients, scheduler and billing.
clinic.py	**Address / ContactInfo**	Value objects for postal address and phone numbers, email and website.
clinic.py	**Department**	A department with location, phone, operating hours and assigned staff.
clinic.py	**Laboratory**	Specialised Department holding the lab test catalogue and incoming lab orders.
clinic.py	**Pharmacy**	Specialised Department holding the medication formulary and received prescriptions.
hours.py	**WeeklyHours**	Opening or working hours per weekday; checks whether a period is covered.
people.py	**Person «abstract»**	Shared personal details (name, date of birth, contacts); abstract role_description().
people.py	**Patient**	Patient profile: emergency contact, insurance, allergies, history summary, account and medical record.
people.py	**EmergencyContact**	Name, relationship and phone of the patient's emergency contact.
people.py	**Staff**	Staff member with role, qualifications, working hours and department.
people.py	**Doctor**	Starts visits, prescribes (with allergy check) and orders lab tests; has speciality and fee.

people.py	**Nurse / LabTechnician / Pharmacist**	Record vitals; collect samples and upload results; dispense prescriptions.
accounts.py	**UserAccount**	Patient login with salted PBKDF2 password hash.
scheduling.py	**AppointmentScheduler**	Searches free slots, books, reschedules and cancels; enforces no double booking.
scheduling.py	**Appointment**	Links a patient and a doctor to a time slot; tracks status and change history.
scheduling.py	**TimeSlot / AvailableSlot**	Start-end period with overlap check; a free slot offered by a doctor.
emr.py	**MedicalRecord**	A patient's collection of visits; applies access control on every read.
emr.py	**AccessPolicy**	Decides who may read a record (staff, or the patient themself).
emr.py	**Visit**	One consultation: notes, vitals, diagnoses, attachments, prescriptions and lab orders; produces the summary.
emr.py	**Vitals / Diagnosis / ClinicalNote / Attachment**	Clinical data recorded in a visit, each with validation.
prescriptions.py	**Prescription**	Issued by a doctor for a visit; DRAFT → SENT_TO_PHARMACY → DISPENSED.
prescriptions.py	**PrescriptionItem / Medication**	Drug, dosage, directions and quantity; formulary item with unit price.
lab.py	**LabOrder**	Test ordered for a visit; ORDERED → SAMPLE_COLLECTED → COMPLETED; validates patient and visit.
lab.py	**LabTest / LabResult**	Catalogue test with price; result values, interpretation and abnormal flag.
billing.py	**BillingService**	Generates one invoice per visit from consultation, lab, medication and other charges.
billing.py	**Invoice / InvoiceLineItem**	Charges, insurance split, co-pay, outstanding balances and status.
billing.py	**InsurancePolicy**	Coverage %, flat co-pay, annual limit, expiry and covered categories.
billing.py	**Payment «abstract»**	Template method process(); subclasses supply _authorise().
billing.py	**CardPayment / MobileWalletPayment / InsuranceClaimPayment**	Concrete payment methods (Luhn check, wallet reference, insurance claim).
enums.py	**Enumerations**	StaffRole, AppointmentStatus, PrescriptionStatus, LabOrderStatus, InvoiceStatus, PaymentStatus, ChargeCategory.
exceptions.py	**ClinicError hierarchy**	ValidationError, NotFoundError, SlotUnavailableError, InvalidStateError, AccessDeniedError, PaymentError.

a.3 Relationships

Inheritance (generalisation) — hollow triangle

Relationship	Rationale
Person ← Patient, Staff	Patients and staff share personal details; Person is abstract so it cannot be created on its own.
Staff ← Doctor, Nurse, LabTechnician, Pharmacist	Each role adds its own behaviour (prescribe, record vitals, upload results, dispense).
Department ← Laboratory, Pharmacy	Both are departments but also keep a catalogue (tests / medications).
Payment ← CardPayment, MobileWalletPayment, InsuranceClaimPayment	Polymorphism: Invoice handles any Payment; each subclass authorises differently.

Composition (filled diamond) — the part cannot exist without the whole

Relationship	Rationale
Clinic ◆— 1..* Department	Departments are created for and owned by one clinic.
Clinic ◆— 1 AppointmentScheduler, BillingService, ContactInfo	Services are created in the Clinic constructor and live and die with it.
Patient ◆— 1 MedicalRecord; 0..1 EmergencyContact	The record is created with the patient and is meaningless without them.
MedicalRecord ◆— 0..* Visit	A visit always belongs to exactly one patient's record.
Visit ◆— Vitals, Diagnosis, ClinicalNote, Attachment	Clinical entries only exist within the visit where they were recorded.
Prescription ◆— 1..* PrescriptionItem; Invoice ◆— 1..* InvoiceLineItem	Lines are part of the document they appear on.
LabOrder ◆— 0..1 LabResult; Appointment ◆— 1 TimeSlot	A result belongs to one order; each appointment owns its slot.
Department / Staff ◆— 1 WeeklyHours	Each has its own hours object, not shared.

Aggregation (hollow diamond) — the part exists independently

Relationship	Rationale
Department ◇— 0..* Staff	Staff can be moved between departments without being deleted.
Clinic ◇— 0..* Patient	Patients are registered with, but are not part of, the clinic.
AppointmentScheduler ◇— 0..* Appointment; BillingService ◇— 0..* Invoice	Services keep collections of independent objects.
Invoice ◇— 0..* Payment	A payment is its own transaction that settles an invoice.
Laboratory ◇— LabTest; Pharmacy ◇— Medication	Catalogue items are reusable reference data.

Association (arrow) — objects use / refer to each other

Relationship	Rationale
Appointment → Patient, Doctor (0..* to 1)	Each appointment is for one patient with one doctor.
Appointment — Visit (1 to 0..1)	A visit results from exactly one appointment.
Visit → Prescription, LabOrder (1 to 0..*)	Orders made during a visit stay linked to it.

Patient → InsurancePolicy, UserAccount	A patient may be insured and has one login account.
Invoice → Visit, InsurancePolicy	One invoice per visit; the policy used for the co-pay calculation.
Prescription → Pharmacy; PrescriptionItem → Medication; LabOrder → LabTest	Links to where/what was ordered.

Dependency (dashed arrow) — temporary use

Relationship	Rationale
AppointmentScheduler ..> AvailableSlot	Creates search results but does not keep them.
MedicalRecord ..> AccessPolicy	Asks the policy on every read.
Doctor ..> Prescription, LabOrder	A doctor creates these objects.
Classes ..> enumerations	Status and role attributes are typed by enums.

a.4 Modularity

Each functional area from the requirements has its own module, so a change to billing rules touches only `billing.py`. Shared building blocks (enumerations, exceptions, utilities, weekly hours) sit at the bottom and depend on nothing else. Domain modules (`scheduling`, `emr`, `prescriptions`, `lab`, `billing`) sit in the middle. `people` and `clinic` sit on top and coordinate them. Where two modules only need each other's types for annotations, the import is placed under `typing.TYPE_CHECKING`, which prevents circular imports at run time. Presentation code (`formatting.py`) is kept separate from domain classes, and test data lives in `sample_data.py` so the demo and the tests share one realistic set-up.

Layer	Modules
Foundation	enums.py, exceptions.py, utils.py, hours.py
Domain	accounts.py, scheduling.py, emr.py, prescriptions.py, lab.py, billing.py
Aggregates / facade	people.py, clinic.py
Presentation / setup	formatting.py, sample_data.py, main.py
Verification	tests/test_clinic_system.py

a.5 Assumptions

- The system manages one clinic. The Clinic class is the single entry point, but no global state is used, so a second clinic could be created.
- Data is held in memory for the assignment. Persistence (a database) would sit behind the same classes and is out of scope.
- Appointment slots are generated from each doctor's weekly working hours in 30-minute steps by default. Other durations are supported.
- A doctor can only start a visit for their own active appointment. One visit is recorded per appointment, and closing the visit marks the appointment COMPLETED.
- Any clinic staff member may view visit summaries. A patient may view only their own record. Anyone else is refused (AccessDeniedError).
- The prescription allergy check matches the patient's recorded allergies against the medication name.

- Insurance rule: eligible charges − flat co-pay, multiplied by the coverage %, capped at the remaining annual limit. The patient pays the rest. Policies can exclude charge categories.
- Insurance claims settle the insurer's share. Card and mobile wallet payments settle the patient's share. Payment gateways are simulated (the card number is Luhn-checked and the expiry date checked).
- Money uses Decimal rounded to cents, never float. The currency symbol is $ for display.
- A Receptionist role exists in StaffRole, but no special behaviour was required, so it has no subclass.

a.6 Design principles applied

- **Encapsulation:** internal collections are private and are returned as copies. State changes go only through methods that validate them (e.g. `Appointment._cancel` is called only by the scheduler).
- **Abstraction and polymorphism:** `Person.role_description()` and `Payment._authorise()` are abstract and implemented differently by each subclass.
- **Template method pattern:** `Payment.process()` fixes the algorithm; subclasses supply only the authorisation step.
- **Facade:** `Clinic` gives one simple entry point to departments, staff, patients, scheduling and billing.
- **Fail fast:** invalid input raises a specific `ClinicError` subclass with a clear message instead of corrupting data.
- **Security:** passwords are salted and hashed (PBKDF2), card numbers are shown masked, and medical-record reads are access-checked.

b. Python Classes

The complete source code follows, file by file. The repository layout is:

```
royal-care-clinic/
├── royal_care/               # application package
│   ├── enums.py              # status / role enumerations
│   ├── exceptions.py         # custom exception hierarchy
│   ├── utils.py              # ID generator, money helpers, dates
│   ├── hours.py              # WeeklyHours (operating / working hours)
│   ├── clinic.py             # Clinic, Department, Laboratory, Pharmacy, ...
│   ├── people.py             # Person, Patient, Staff, Doctor, Nurse, ...
│   ├── accounts.py           # UserAccount (patient login)
│   ├── scheduling.py         # TimeSlot, Appointment, AppointmentScheduler
│   ├── emr.py                # MedicalRecord, Visit, Vitals, Diagnosis, ...
│   ├── prescriptions.py      # Medication, Prescription, PrescriptionItem
│   ├── lab.py                # LabTest, LabOrder, LabResult
│   ├── billing.py            # Invoice, InsurancePolicy, Payment subclasses
│   ├── formatting.py         # console output helpers
│   └── sample_data.py        # builds the sample Royal Care Clinic
├── tests/
│   └── test_clinic_system.py # unit + scenario tests
├── docs/
│   ├── uml_class_diagram.puml # UML source (PlantUML)
│   ├── uml_class_diagram.png  # rendered diagram
│   └── uml_class_diagram.svg  # scalable version
└── main.py                   # end-to-end demonstration
```

`royal_care/__init__.py`

Package initialiser and public API

```
"""
Royal Care Clinic Management System
==================================
Object-oriented model of an outpatient clinic: clinic information,
patients, staff, appointments, electronic medical records, prescriptions,
laboratory orders and billing.

The most commonly used classes are re-exported here so that callers can
simply write ``from royal_care import Clinic, Patient, Doctor``.
"""

from .accounts import UserAccount
from .billing import (BillingService, CardPayment, InsuranceClaimPayment,
                      InsurancePolicy, Invoice, InvoiceLineItem,
                      MobileWalletPayment, Payment)
from .clinic import Address, Clinic, ContactInfo, Department, Laboratory, Pharmacy
from .emr import AccessPolicy, Attachment, Diagnosis, MedicalRecord, Visit, Vitals
from .enums import (AppointmentStatus, ChargeCategory, InvoiceStatus, LabOrderStatus,
                    PaymentStatus, PrescriptionStatus, StaffRole)
from .exceptions import (AccessDeniedError, ClinicError, InvalidStateError,
                         NotFoundError, PaymentError, SlotUnavailableError,
                         ValidationError)
from .hours import WeeklyHours
from .lab import LabOrder, LabResult, LabTest
from .people import (Doctor, EmergencyContact, LabTechnician, Nurse, Patient,
                     Person, Pharmacist, Staff)
from .prescriptions import Medication, Prescription, PrescriptionItem
from .scheduling import Appointment, AppointmentScheduler, AvailableSlot, TimeSlot

__all__ = [name for name in dir() if not name.startswith("_")]
__version__ = "1.0.0"
```

`royal_care/enums.py`

Enumerations

```
"""
enums.py
```

```
========
Enumerations shared across the Royal Care Clinic Management System.

Using ``Enum`` instead of raw strings means a typo such as ``"COMPLETD"``
is caught straight away, and every legal state of an object is listed
in one place.
"""

from enum import Enum


class StaffRole(Enum):
    """Roles a member of staff can hold in the clinic."""

    DOCTOR = "Doctor"
    NURSE = "Nurse"
    LAB_TECHNICIAN = "Lab Technician"
    PHARMACIST = "Pharmacist"
    RECEPTIONIST = "Receptionist"


class AppointmentStatus(Enum):
    """Lifecycle of an appointment."""

    BOOKED = "BOOKED"
    RESCHEDULED = "RESCHEDULED"
    CANCELLED = "CANCELLED"
    COMPLETED = "COMPLETED"


class PrescriptionStatus(Enum):
    """Lifecycle of a prescription."""

    DRAFT = "DRAFT"
    SENT_TO_PHARMACY = "SENT_TO_PHARMACY"
    DISPENSED = "DISPENSED"


class LabOrderStatus(Enum):
    """Lifecycle of a laboratory order."""

    ORDERED = "ORDERED"
    SAMPLE_COLLECTED = "SAMPLE_COLLECTED"
    COMPLETED = "COMPLETED"
    CANCELLED = "CANCELLED"


class InvoiceStatus(Enum):
    """Payment state of an invoice."""

    UNPAID = "UNPAID"
    PARTIALLY_PAID = "PARTIALLY_PAID"
    PAID = "PAID"


class PaymentStatus(Enum):
    """Outcome of a single payment attempt."""

    PENDING = "PENDING"
    SUCCESSFUL = "SUCCESSFUL"
    FAILED = "FAILED"


class ChargeCategory(Enum):
    """Category of a line on an invoice."""

    CONSULTATION = "Consultation"
    LAB = "Laboratory"
    MEDICATION = "Medication"
    OTHER = "Other Service"
```

`royal_care/exceptions.py`

Custom exception hierarchy

```
"""
exceptions.py
=============
Custom exception hierarchy for the clinic system.

All exceptions inherit from :class:`ClinicError`, so a caller can catch
every domain error with a single ``except ClinicError`` clause while
still being able to handle specific cases individually.
"""


class ClinicError(Exception):
    """Base class for every error raised by the clinic system."""


class ValidationError(ClinicError):
    """Raised when input data is missing or invalid."""


class NotFoundError(ClinicError):
    """Raised when a requested record (patient, doctor, ...) does not exist."""


class SlotUnavailableError(ClinicError):
    """Raised when an appointment slot is already taken or outside working hours."""


class InvalidStateError(ClinicError):
    """Raised when an operation is not allowed in an object's current state."""


class AccessDeniedError(ClinicError):
    """Raised when a user tries to view medical data they are not allowed to see."""


class PaymentError(ClinicError):
    """Raised when a payment cannot be processed."""
```

royal_care/utils.py

Utility helpers

```
"""
utils.py
========
Small helper functions used by several modules:

* ``IdGenerator``  - produces readable, unique identifiers (e.g. ``PAT-0001``).
* ``to_money``     - converts numbers to ``Decimal`` rounded to 2 places.
* ``next_weekday`` - finds the next given weekday (used for "next Monday").
* ``format_money`` - formats a Decimal as a currency string.
"""

from datetime import date, timedelta
from decimal import Decimal, ROUND_HALF_UP
from itertools import count
from typing import Dict, Iterator, Union

Number = Union[int, float, str, Decimal]

CURRENCY = "$"


class IdGenerator:
    """Generates sequential, prefixed identifiers such as ``APT-0003``.

    One counter is kept per prefix, so patients and appointments are
    numbered independently.
    """

    _counters: Dict[str, Iterator[int]] = {}

    @classmethod
    def next_id(cls, prefix: str) -> str:
```

```
        """Return the next identifier for ``prefix``."""
        if prefix not in cls._counters:
            cls._counters[prefix] = count(1)
        return f"{prefix}-{next(cls._counters[prefix]):04d}"

    @classmethod
    def reset(cls) -> None:
        """Reset all counters (useful to make tests independent)."""
        cls._counters.clear()


def to_money(value: Number) -> Decimal:
    """Convert ``value`` to a Decimal rounded to cents.

    ``Decimal`` is used instead of ``float`` because floating point
    numbers cannot represent most cent values exactly (0.1 + 0.2 != 0.3).
    """
    return Decimal(str(value)).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)


def format_money(value: Decimal) -> str:
    """Return ``value`` formatted as currency, e.g. ``$1,250.00``."""
    return f"{CURRENCY}{value:,.2f}"


def next_weekday(start: date, weekday: int) -> date:
    """Return the next date after ``start`` falling on ``weekday`` (Mon=0).

    If ``start`` is itself that weekday, the date one week later is
    returned, which matches the everyday meaning of "next Monday".
    """
    days_ahead = (weekday - start.weekday()) % 7 or 7
    return start + timedelta(days=days_ahead)
```

royal_care/hours.py

WeeklyHours

```
"""
hours.py
========
:class:`WeeklyHours` stores opening or working hours for each day of the week.

It is used both for department operating hours and for the working hours
of individual doctors and staff, so the logic lives in one place.
"""

from datetime import datetime, time
from typing import Dict, List, Optional, Tuple

from .exceptions import ValidationError

DAY_NAMES = ["Monday", "Tuesday", "Wednesday", "Thursday",
             "Friday", "Saturday", "Sunday"]


class WeeklyHours:
    """Opening/working hours keyed by weekday (Monday = 0 ... Sunday = 6)."""

    def __init__(self) -> None:
        # Private dictionary: weekday -> (start time, end time)
        self._hours: Dict[int, Tuple[time, time]] = {}

    # ------------------------------------------------------------------ #
    # Factory helpers
    # ------------------------------------------------------------------ #
    @classmethod
    def weekdays(cls, start: str, end: str) -> "WeeklyHours":
        """Create hours that repeat Monday to Friday, e.g. ``("08:00", "17:00")``."""
        hours = cls()
        for day in range(5):
            hours.set_day(day, start, end)
        return hours
```

```
    # ------------------------------------------------------------------ #
    # Commands
    # ------------------------------------------------------------------ #
    def set_day(self, weekday: int, start: str, end: str) -> None:
        """Set hours for one weekday using ``"HH:MM"`` strings."""
        if weekday not in range(7):
            raise ValidationError("Weekday must be between 0 (Mon) and 6 (Sun).")
        start_t = time.fromisoformat(start)
        end_t = time.fromisoformat(end)
        if start_t >= end_t:
            raise ValidationError("Start time must be earlier than end time.")
        self._hours[weekday] = (start_t, end_t)

    def close_day(self, weekday: int) -> None:
        """Mark a weekday as closed / not working."""
        self._hours.pop(weekday, None)

    # ------------------------------------------------------------------ #
    # Queries
    # ------------------------------------------------------------------ #
    def hours_for(self, weekday: int) -> Optional[Tuple[time, time]]:
        """Return ``(start, end)`` for the weekday or ``None`` if closed."""
        return self._hours.get(weekday)

    def covers(self, start: datetime, end: datetime) -> bool:
        """Return True if the whole period ``start``-``end`` lies within the hours."""
        if start.date() != end.date():
            return False
        day_hours = self.hours_for(start.weekday())
        if day_hours is None:
            return False
        open_t, close_t = day_hours
        return open_t <= start.time() and end.time() <= close_t

    def describe(self) -> List[str]:
        """Return human-readable lines such as ``"Monday: 08:00-17:00"``."""
        lines = []
        for day, name in enumerate(DAY_NAMES):
            span = self._hours.get(day)
            text = f"{span[0]:%H:%M}-{span[1]:%H:%M}" if span else "Closed"
            lines.append(f"{name}: {text}")
        return lines

    def __repr__(self) -> str:
        return f"WeeklyHours({len(self._hours)} working days)"
```

royal_care/clinic.py

Clinic information and departments

```
"""
clinic.py
=========
Clinic information: the clinic profile, its address and contact details,
and its departments.

Relationships implemented here
------------------------------
* ``Clinic`` *composition* ``Department``  - departments are created for
  and owned by one clinic; they do not exist on their own.
* ``Department`` *aggregation* ``Staff``   - staff are assigned to a
  department but exist independently of it (they can be moved).
* ``Laboratory`` and ``Pharmacy`` *inherit* from ``Department`` and add
  their own catalogues (lab tests / medications).
* ``Clinic`` *composition* ``AppointmentScheduler`` and ``BillingService``.
"""

from __future__ import annotations

from datetime import date
from typing import TYPE_CHECKING, Dict, List, Optional

from .accounts import UserAccount
from .billing import BillingService
```

```python
from .exceptions import NotFoundError, ValidationError
from .hours import WeeklyHours
from .people import Doctor, EmergencyContact, Patient, Staff
from .scheduling import AppointmentScheduler

if TYPE_CHECKING:  # imported only for type hints, avoids circular imports
    from .billing import InsurancePolicy
    from .lab import LabOrder, LabTest
    from .prescriptions import Medication, Prescription


# ============================================================== #
# Value objects
# ============================================================== #
class Address:
    """Postal address of the clinic or of a person."""

    def __init__(self, street: str, city: str, country: str,
                 postal_code: str = "") -> None:
        self.street = street
        self.city = city
        self.country = country
        self.postal_code = postal_code

    def __str__(self) -> str:
        parts = [self.street, self.city, self.postal_code, self.country]
        return ", ".join(p for p in parts if p)


class ContactInfo:
    """Phone numbers, email and website of the clinic."""

    def __init__(self, phone_numbers: List[str], email: str,
                 website: str = "") -> None:
        if not phone_numbers:
            raise ValidationError("At least one phone number is required.")
        if "@" not in email:
            raise ValidationError(f"Invalid email address: {email!r}")
        self.phone_numbers = list(phone_numbers)
        self.email = email
        self.website = website

    def __str__(self) -> str:
        return (f"Tel: {', '.join(self.phone_numbers)} | Email: {self.email}"
                f" | Web: {self.website or 'n/a'}")


# ============================================================== #
# Departments
# ============================================================== #
class Department:
    """A clinical or service department such as Cardiology or Radiology."""

    def __init__(self, name: str, description: str = "",
                 location: str = "", phone: str = "",
                 operating_hours: Optional[WeeklyHours] = None) -> None:
        if not name.strip():
            raise ValidationError("Department name cannot be empty.")
        self._name = name
        self.description = description
        self.location = location
        self.phone = phone
        # Composition: each department owns its own operating hours object
        self.operating_hours = operating_hours or WeeklyHours.weekdays("08:00", "17:00")
        # Aggregation: staff members exist independently of the department
        self._staff: List[Staff] = []

    @property
    def name(self) -> str:
        """Department name (read-only)."""
        return self._name

    @property
    def staff(self) -> List[Staff]:
        """A copy of the staff list so callers cannot modify it directly."""
        return list(self._staff)
```

```python
    def add_staff(self, member: Staff) -> None:
        """Assign a staff member to this department."""
        if member not in self._staff:
            self._staff.append(member)
            member.department = self

    def remove_staff(self, member: Staff) -> None:
        """Remove a staff member (the staff object itself continues to exist)."""
        if member in self._staff:
            self._staff.remove(member)
            member.department = None

    def __repr__(self) -> str:
        return f"{type(self).__name__}({self._name!r}, staff={len(self._staff)})"


class Laboratory(Department):
    """Laboratory department: keeps a test catalogue and receives lab orders."""

    def __init__(self, name: str = "Laboratory", **kwargs) -> None:
        super().__init__(name, **kwargs)
        self._test_catalogue: Dict[str, LabTest] = {}
        self._orders: List[LabOrder] = []

    def add_test(self, test: LabTest) -> None:
        """Add a test (e.g. Full Blood Count) to the catalogue."""
        self._test_catalogue[test.code] = test

    def get_test(self, code: str) -> LabTest:
        """Look up a test by its code."""
        try:
            return self._test_catalogue[code]
        except KeyError:
            raise NotFoundError(f"Lab test {code!r} is not offered.") from None

    @property
    def tests(self) -> List[LabTest]:
        return list(self._test_catalogue.values())

    def receive_order(self, order: LabOrder) -> None:
        """Queue a lab order sent by a doctor."""
        self._orders.append(order)

    def pending_orders(self) -> List[LabOrder]:
        """Orders that do not yet have a result."""
        return [o for o in self._orders if not o.is_complete]


class Pharmacy(Department):
    """Pharmacy department: keeps a formulary and receives prescriptions."""

    def __init__(self, name: str = "Pharmacy", **kwargs) -> None:
        super().__init__(name, **kwargs)
        self._formulary: Dict[str, Medication] = {}
        self._received: List[Prescription] = []

    def add_medication(self, medication: Medication) -> None:
        """Add a medication to the pharmacy formulary."""
        self._formulary[medication.name.lower()] = medication

    def get_medication(self, name: str) -> Medication:
        """Look up a medication by name (case-insensitive)."""
        try:
            return self._formulary[name.lower()]
        except KeyError:
            raise NotFoundError(f"Medication {name!r} is not stocked.") from None

    def receive_prescription(self, prescription: Prescription) -> None:
        """Called by ``Prescription.send_to_pharmacy``."""
        self._received.append(prescription)

    @property
    def received_prescriptions(self) -> List[Prescription]:
        return list(self._received)


# ============================================================================ #
```

```python
# Clinic (aggregate root / facade)
# ============================================================================ #
class Clinic:
    """The clinic profile and the entry point to all clinic services."""

    _EDITABLE_FIELDS = {"name", "address", "contact"}

    def __init__(self, name: str, address: Address, contact: ContactInfo) -> None:
        self._name = name
        self._address = address
        self._contact = contact
        self._departments: Dict[str, Department] = {}   # composition
        self._staff: Dict[str, Staff] = {}
        self._patients: Dict[str, Patient] = {}
        self._accounts: Dict[str, UserAccount] = {}
        # Composition: services are created with and owned by the clinic
        self.scheduler = AppointmentScheduler(self)
        self.billing = BillingService()

    # ---------------------------- profile ----------------------------- #
    @property
    def name(self) -> str:
        return self._name

    @property
    def address(self) -> Address:
        return self._address

    @property
    def contact(self) -> ContactInfo:
        return self._contact

    def update_info(self, **changes) -> None:
        """Update clinic profile fields: ``name``, ``address`` or ``contact``."""
        unknown = set(changes) - self._EDITABLE_FIELDS
        if unknown:
            raise ValidationError(f"Cannot update field(s): {', '.join(sorted(unknown))}")
        for field, value in changes.items():
            setattr(self, f"_{field}", value)

    def get_info(self) -> Dict[str, object]:
        """Return the clinic profile as a dictionary (easy to display or export)."""
        return {
            "name": self._name,
            "address": str(self._address),
            "phone_numbers": self._contact.phone_numbers,
            "email": self._contact.email,
            "website": self._contact.website,
            "departments": {
                d.name: d.operating_hours.describe() for d in self._departments.values()
            },
        }

    # ------------------------- departments ---------------------------- #
    def add_department(self, department: Department) -> Department:
        """Add a department; names must be unique within the clinic."""
        key = department.name.lower()
        if key in self._departments:
            raise ValidationError(f"Department {department.name!r} already exists.")
        self._departments[key] = department
        return department

    def get_department(self, name: str) -> Department:
        try:
            return self._departments[name.lower()]
        except KeyError:
            raise NotFoundError(f"Department {name!r} not found.") from None

    def update_department(self, name: str, **changes) -> Department:
        """Update description, location, phone or operating_hours of a department."""
        department = self.get_department(name)
        allowed = {"description", "location", "phone", "operating_hours"}
        unknown = set(changes) - allowed
        if unknown:
            raise ValidationError(f"Cannot update field(s): {', '.join(sorted(unknown))}")
        for field, value in changes.items():
            setattr(department, field, value)
```

```
        return department

    @property
    def departments(self) -> List[Department]:
        return list(self._departments.values())

    def laboratory(self) -> Laboratory:
        """Return the clinic's laboratory department."""
        return self._first_of_type(Laboratory)

    def pharmacy(self) -> Pharmacy:
        """Return the clinic's pharmacy department."""
        return self._first_of_type(Pharmacy)

    def _first_of_type(self, kind: type) -> Department:
        for dept in self._departments.values():
            if isinstance(dept, kind):
                return dept
        raise NotFoundError(f"The clinic has no {kind.__name__} department.")

    # ---------------------------- staff ------------------------------ #
    def add_staff(self, member: Staff, department_name: str) -> Staff:
        """Register a staff member and assign them to a department."""
        self.get_department(department_name).add_staff(member)
        self._staff[member.person_id] = member
        return member

    def get_staff(self, staff_id: str) -> Staff:
        try:
            return self._staff[staff_id]
        except KeyError:
            raise NotFoundError(f"Staff member {staff_id!r} not found.") from None

    def doctors(self, speciality: Optional[str] = None) -> List[Doctor]:
        """All doctors, optionally filtered by speciality (case-insensitive)."""
        result = [s for s in self._staff.values() if isinstance(s, Doctor)]
        if speciality:
            result = [d for d in result if d.speciality.lower() == speciality.lower()]
        return result

    # ---------------------------- patients ---------------------------- #
    def register_patient(self, first_name: str, last_name: str,
                         date_of_birth: date, phone: str, email: str,
                         address: Address, username: str, password: str,
                         emergency_contact: Optional[EmergencyContact] = None,
                         insurance: Optional[InsurancePolicy] = None,
                         allergies: Optional[List[str]] = None,
                         medical_history_summary: str = "") -> Patient:
        """Create a patient profile together with a login account."""
        if username.lower() in self._accounts:
            raise ValidationError(f"Username {username!r} is already taken.")
        patient = Patient(first_name, last_name, date_of_birth, phone, email,
                          address, emergency_contact, insurance,
                          allergies, medical_history_summary)
        account = UserAccount(username, password, owner=patient)
        patient.account = account
        self._accounts[username.lower()] = account
        self._patients[patient.person_id] = patient
        return patient

    def login(self, username: str, password: str) -> Patient:
        """Return the patient if the credentials are valid."""
        account = self._accounts.get(username.lower())
        if account is None or not account.verify_password(password):
            raise ValidationError("Invalid username or password.")
        return account.owner

    def get_patient(self, patient_id: str) -> Patient:
        try:
            return self._patients[patient_id]
        except KeyError:
            raise NotFoundError(f"Patient {patient_id!r} not found.") from None

    @property
    def patients(self) -> List[Patient]:
        return list(self._patients.values())
```

```
    def __repr__(self) -> str:
        return (f"Clinic({self._name!r}, departments={len(self._departments)}, "
                f"staff={len(self._staff)}, patients={len(self._patients)})")
```

royal_care/accounts.py

Patient login accounts

```
"""
accounts.py
===========
:class:`UserAccount` - the login account a patient creates to use the
patient portal (view appointments, update profile, download summaries).

Passwords are never stored in plain text: a random salt is generated and
the password is hashed with PBKDF2-HMAC-SHA256 from the standard library.
"""

from __future__ import annotations

import hashlib
import hmac
import os
from datetime import datetime
from typing import TYPE_CHECKING

from .exceptions import ValidationError

if TYPE_CHECKING:
    from .people import Person

MIN_PASSWORD_LENGTH = 8
_ITERATIONS = 100_000


class UserAccount:
    """Login credentials linked to one person (one-to-one association)."""

    def __init__(self, username: str, password: str, owner: Person) -> None:
        if len(username.strip()) < 3:
            raise ValidationError("Username must be at least 3 characters.")
        self._username = username.strip()
        self._salt = os.urandom(16)
        self._password_hash = self._hash(password)
        self.owner = owner
        self.created_at = datetime.now()
        self.is_active = True

    @property
    def username(self) -> str:
        return self._username

    # ------------------------------------------------------------------ #
    def _hash(self, password: str) -> bytes:
        """Validate the password and return its salted hash (private helper)."""
        if len(password) < MIN_PASSWORD_LENGTH:
            raise ValidationError(
                f"Password must be at least {MIN_PASSWORD_LENGTH} characters.")
        return hashlib.pbkdf2_hmac("sha256", password.encode(), self._salt, _ITERATIONS)

    def verify_password(self, password: str) -> bool:
        """Return True if ``password`` matches (constant-time comparison)."""
        if not self.is_active:
            return False
        candidate = hashlib.pbkdf2_hmac("sha256", password.encode(),
                                        self._salt, _ITERATIONS)
        return hmac.compare_digest(candidate, self._password_hash)

    def change_password(self, old_password: str, new_password: str) -> None:
        """Change the password after confirming the old one."""
        if not self.verify_password(old_password):
            raise ValidationError("Current password is incorrect.")
        self._password_hash = self._hash(new_password)
```

```
    def __repr__(self) -> str:
        return f"UserAccount({self._username!r}, active={self.is_active})"
```

royal_care/people.py

Person hierarchy

```
"""
people.py
==========
People in the clinic: patients and staff.

Inheritance hierarchy
---------------------
::

    Person (abstract)
     ├── Patient
     └── Staff
          ├── Doctor
          ├── Nurse
          ├── LabTechnician
          └── Pharmacist

* ``Patient`` *composition* ``MedicalRecord``  - the record is created with
  the patient and belongs only to that patient.
* ``Patient`` *composition* ``EmergencyContact``.
* ``Patient`` *association* ``InsurancePolicy`` and ``UserAccount``.
* ``Staff`` *composition* ``WeeklyHours`` (working hours).
"""

from __future__ import annotations

import re
from abc import ABC, abstractmethod
from datetime import date, datetime
from pathlib import Path
from typing import TYPE_CHECKING, Dict, List, Optional

from .emr import MedicalRecord, Visit, Vitals
from .enums import StaffRole
from .exceptions import InvalidStateError, ValidationError
from .hours import WeeklyHours
from .lab import LabOrder, LabResult, LabTest
from .prescriptions import Prescription, PrescriptionItem
from .utils import IdGenerator, to_money

if TYPE_CHECKING:
    from .accounts import UserAccount
    from .billing import InsurancePolicy
    from .clinic import Address, Department, Laboratory
    from .scheduling import Appointment, AppointmentScheduler

_EMAIL_PATTERN = re.compile(r"^[^@\s]+@[^@\s]+\.[^@\s]+$")


# ============================================================== #
# Base class
# ============================================================== #
class Person(ABC):
    """Abstract base class holding personal details shared by all people."""

    _ID_PREFIX = "PER"

    def __init__(self, first_name: str, last_name: str, date_of_birth: date,
                 phone: str, email: str, address: Optional[Address] = None) -> None:
        if not first_name.strip() or not last_name.strip():
            raise ValidationError("First and last name are required.")
        if date_of_birth > date.today():
            raise ValidationError("Date of birth cannot be in the future.")
        if not _EMAIL_PATTERN.match(email):
            raise ValidationError(f"Invalid email address: {email!r}")
        self._person_id = IdGenerator.next_id(self._ID_PREFIX)
        self._first_name = first_name.strip()
```

```
        self._last_name = last_name.strip()
        self._date_of_birth = date_of_birth
        self.phone = phone
        self._email = email
        self.address = address

    # ---- read-only / validated properties ---- #
    @property
    def person_id(self) -> str:
        return self._person_id

    @property
    def first_name(self) -> str:
        return self._first_name

    @property
    def last_name(self) -> str:
        return self._last_name

    @property
    def full_name(self) -> str:
        return f"{self._first_name} {self._last_name}"

    @property
    def date_of_birth(self) -> date:
        return self._date_of_birth

    @property
    def email(self) -> str:
        return self._email

    @email.setter
    def email(self, value: str) -> None:
        if not _EMAIL_PATTERN.match(value):
            raise ValidationError(f"Invalid email address: {value!r}")
        self._email = value

    def age(self, on: Optional[date] = None) -> int:
        """Age in whole years on the given date (default: today)."""
        on = on or date.today()
        had_birthday = (on.month, on.day) >= (self._date_of_birth.month,
                                              self._date_of_birth.day)
        return on.year - self._date_of_birth.year - (0 if had_birthday else 1)

    @abstractmethod
    def role_description(self) -> str:
        """Short description of the person's role (implemented by subclasses)."""

    def __str__(self) -> str:
        return f"{self.full_name} ({self.role_description()}, {self._person_id})"


# ============================================================================ #
# Patient
# ============================================================================ #
class EmergencyContact:
    """Person to contact in an emergency (owned by a Patient)."""

    def __init__(self, name: str, relationship: str, phone: str) -> None:
        self.name = name
        self.relationship = relationship
        self.phone = phone

    def __str__(self) -> str:
        return f"{self.name} ({self.relationship}) - {self.phone}"


class Patient(Person):
    """A patient registered with the clinic."""

    _ID_PREFIX = "PAT"
    _EDITABLE_FIELDS = {"phone", "email", "address", "emergency_contact",
                        "insurance", "medical_history_summary"}

    def __init__(self, first_name: str, last_name: str, date_of_birth: date,
                 phone: str, email: str, address: Optional[Address] = None,
                 emergency_contact: Optional[EmergencyContact] = None,
```

```
                 insurance: Optional[InsurancePolicy] = None,
                 allergies: Optional[List[str]] = None,
                 medical_history_summary: str = "") -> None:
        super().__init__(first_name, last_name, date_of_birth, phone, email, address)
        self.emergency_contact = emergency_contact
        self.insurance = insurance
        self._allergies: List[str] = [a.strip() for a in (allergies or [])]
        self.medical_history_summary = medical_history_summary
        self.account: Optional[UserAccount] = None
        # Composition: the medical record is created with (and owned by) the patient
        self._medical_record = MedicalRecord(self)

    def role_description(self) -> str:
        return "Patient"

    @property
    def medical_record(self) -> MedicalRecord:
        return self._medical_record

    @property
    def allergies(self) -> List[str]:
        return list(self._allergies)

    def add_allergy(self, allergy: str) -> None:
        if allergy.strip() and not self.is_allergic_to(allergy):
            self._allergies.append(allergy.strip())

    def is_allergic_to(self, substance: str) -> bool:
        """Case-insensitive check used before prescribing medication."""
        return any(a.lower() == substance.lower() for a in self._allergies)

    def update_profile(self, **changes) -> None:
        """Update editable profile fields (contact details, insurance, etc.)."""
        unknown = set(changes) - self._EDITABLE_FIELDS
        if unknown:
            raise ValidationError(f"Cannot update field(s): {', '.join(sorted(unknown))}")
        for field, value in changes.items():
            setattr(self, field, value)   # email goes through its validating setter

    def upcoming_appointments(self, scheduler: AppointmentScheduler) -> List[Appointment]:
        """Return this patient's future, active appointments in date order."""
        return scheduler.appointments_for_patient(self, upcoming_only=True)

    def download_visit_summary(self, visit_id: str, directory: str = "output") -> Path:
        """Write a visit summary to a text file and return its path.

        Access control is applied by the medical record: a patient can only
        download summaries from their own record.
        """
        visit = self._medical_record.get_visit(visit_id, requester=self)
        folder = Path(directory)
        folder.mkdir(parents=True, exist_ok=True)
        path = folder / f"visit_summary_{visit.visit_id}.txt"
        path.write_text(visit.summary(), encoding="utf-8")
        return path


# ============================================================================ #
# Staff hierarchy
# ============================================================================ #
class Staff(Person):
    """A member of clinic staff."""

    _ID_PREFIX = "STF"

    def __init__(self, first_name: str, last_name: str, date_of_birth: date,
                 phone: str, email: str, role: StaffRole,
                 qualifications: Optional[List[str]] = None,
                 working_hours: Optional[WeeklyHours] = None,
                 address: Optional[Address] = None) -> None:
        super().__init__(first_name, last_name, date_of_birth, phone, email, address)
        self._role = role
        self.qualifications = list(qualifications or [])
        # Composition: each staff member owns their own working-hours object
        self.working_hours = working_hours or WeeklyHours.weekdays("09:00", "17:00")
        self.department: Optional[Department] = None   # set by Department.add_staff
```

```python
    @property
    def role(self) -> StaffRole:
        return self._role

    def role_description(self) -> str:
        return self._role.value

    def is_working(self, start: datetime, end: datetime) -> bool:
        """True if the period falls inside this person's working hours."""
        return self.working_hours.covers(start, end)


class Doctor(Staff):
    """A doctor who conducts visits, diagnoses, prescribes and orders lab tests."""

    _ID_PREFIX = "DOC"

    def __init__(self, first_name: str, last_name: str, date_of_birth: date,
                 phone: str, email: str, speciality: str, license_number: str,
                 consultation_fee, qualifications: Optional[List[str]] = None,
                 working_hours: Optional[WeeklyHours] = None) -> None:
        super().__init__(first_name, last_name, date_of_birth, phone, email,
                         StaffRole.DOCTOR, qualifications, working_hours)
        self.speciality = speciality
        self._license_number = license_number
        self.consultation_fee = to_money(consultation_fee)

    @property
    def license_number(self) -> str:
        return self._license_number

    @property
    def full_name(self) -> str:
        return f"Dr. {self.first_name} {self.last_name}"

    def role_description(self) -> str:
        return f"Doctor - {self.speciality}"

    # ---------------------- clinical actions ---------------------- #
    def start_visit(self, appointment: Appointment) -> Visit:
        """Open a visit (EMR entry) for one of this doctor's appointments."""
        if appointment.doctor is not self:
            raise InvalidStateError("A doctor can only start visits for their own appointments.")
        if not appointment.is_active:
            raise InvalidStateError(
                f"Cannot start a visit for a {appointment.status.value} appointment.")
        if appointment.visit is not None:
            raise InvalidStateError("A visit has already been recorded for this appointment.")
        visit = Visit(appointment)
        appointment.visit = visit
        appointment.patient.medical_record.add_visit(visit)
        return visit

    def prescribe(self, visit: Visit, items: List[PrescriptionItem]) -> Prescription:
        """Create a prescription for the visit, checking the patient's allergies."""
        self._check_own_visit(visit)
        for item in items:
            if visit.patient.is_allergic_to(item.medication.name):
                raise ValidationError(
                    f"{visit.patient.full_name} is allergic to {item.medication.name}.")
        prescription = Prescription(visit, self, items)
        visit.add_prescription(prescription)
        return prescription

    def order_lab_test(self, visit: Visit, test: LabTest,
                       laboratory: Laboratory, notes: str = "") -> LabOrder:
        """Order a lab test for the visit and send it to the laboratory."""
        self._check_own_visit(visit)
        order = LabOrder(visit, test, self, notes)
        visit.add_lab_order(order)
        laboratory.receive_order(order)
        return order

    def _check_own_visit(self, visit: Visit) -> None:
        if visit.doctor is not self:
            raise InvalidStateError("Doctors can only act on their own visits.")
        if visit.is_closed:
```

```
            raise InvalidStateError("The visit is closed and can no longer be changed.")


class Nurse(Staff):
    """A nurse who records patient vitals."""

    _ID_PREFIX = "NUR"

    def __init__(self, first_name: str, last_name: str, date_of_birth: date,
                 phone: str, email: str, **kwargs) -> None:
        super().__init__(first_name, last_name, date_of_birth, phone, email,
                         StaffRole.NURSE, **kwargs)

    def record_vitals(self, visit: Visit, vitals: Vitals) -> None:
        visit.record_vitals(vitals, recorded_by=self)


class LabTechnician(Staff):
    """A laboratory technician who collects samples and uploads results."""

    _ID_PREFIX = "LAB"

    def __init__(self, first_name: str, last_name: str, date_of_birth: date,
                 phone: str, email: str, **kwargs) -> None:
        super().__init__(first_name, last_name, date_of_birth, phone, email,
                         StaffRole.LAB_TECHNICIAN, **kwargs)

    def collect_sample(self, order: LabOrder) -> None:
        order.collect_sample()

    def upload_result(self, order: LabOrder, patient: Patient, visit_id: str,
                      values: Dict[str, str], interpretation: str,
                      is_abnormal: bool = False) -> LabResult:
        """Attach a result to an order after checking patient and visit match."""
        result = LabResult(values, interpretation, is_abnormal, recorded_by=self)
        order.record_result(result, patient=patient, visit_id=visit_id)
        return result


class Pharmacist(Staff):
    """A pharmacist who dispenses prescriptions received by the pharmacy."""

    _ID_PREFIX = "PHA"

    def __init__(self, first_name: str, last_name: str, date_of_birth: date,
                 phone: str, email: str, **kwargs) -> None:
        super().__init__(first_name, last_name, date_of_birth, phone, email,
                         StaffRole.PHARMACIST, **kwargs)

    def dispense(self, prescription: Prescription) -> None:
        prescription.mark_dispensed(self)
```

`royal_care/scheduling.py`

Appointment scheduling

```
"""
scheduling.py
=============
Appointment scheduling: searching for free slots, booking, rescheduling
and cancelling.

* ``TimeSlot``             - a start/end period (value object).
* ``AvailableSlot``        - a free slot offered by a specific doctor.
* ``Appointment``          - associates one Patient with one Doctor at a TimeSlot.
* ``AppointmentScheduler`` - service owned by the Clinic (composition) that
  keeps all appointments and enforces the booking rules.
"""

from __future__ import annotations

from datetime import date, datetime, timedelta
from typing import TYPE_CHECKING, Dict, List, Optional
```

```python
from .enums import AppointmentStatus
from .exceptions import (InvalidStateError, NotFoundError, SlotUnavailableError,
                         ValidationError)
from .utils import IdGenerator

if TYPE_CHECKING:
    from .clinic import Clinic
    from .emr import Visit
    from .people import Doctor, Patient

DEFAULT_SLOT_MINUTES = 30


# ============================================================== #
# Value objects
# ============================================================== #
class TimeSlot:
    """An immutable period of time with a start and an end."""

    def __init__(self, start: datetime, end: datetime) -> None:
        if end <= start:
            raise ValidationError("A time slot must end after it starts.")
        self._start = start
        self._end = end

    @classmethod
    def of_minutes(cls, start: datetime, minutes: int = DEFAULT_SLOT_MINUTES) -> "TimeSlot":
        return cls(start, start + timedelta(minutes=minutes))

    @property
    def start(self) -> datetime:
        return self._start

    @property
    def end(self) -> datetime:
        return self._end

    @property
    def duration_minutes(self) -> int:
        return int((self._end - self._start).total_seconds() // 60)

    def overlaps(self, other: "TimeSlot") -> bool:
        """True if the two slots share any time."""
        return self._start < other.end and other.start < self._end

    def __eq__(self, other: object) -> bool:
        return (isinstance(other, TimeSlot)
                and (self._start, self._end) == (other.start, other.end))

    def __hash__(self) -> int:
        return hash((self._start, self._end))

    def __str__(self) -> str:
        return f"{self._start:%a %d %b %Y %H:%M}-{self._end:%H:%M}"


class AvailableSlot:
    """A free time slot offered by a particular doctor (search result)."""

    def __init__(self, doctor: Doctor, slot: TimeSlot) -> None:
        self.doctor = doctor
        self.slot = slot

    def __repr__(self) -> str:
        return f"AvailableSlot({self.doctor.full_name}, {self.slot})"


# ============================================================== #
# Appointment
# ============================================================== #
class Appointment:
    """A booking of a patient with a doctor for a time slot."""

    def __init__(self, patient: Patient, doctor: Doctor, slot: TimeSlot,
                 reason: str = "") -> None:
        self._appointment_id = IdGenerator.next_id("APT")
        self._patient = patient
```

```
        self._doctor = doctor
        self._slot = slot
        self.reason = reason
        self._status = AppointmentStatus.BOOKED
        self.created_at = datetime.now()
        self.cancellation_reason: Optional[str] = None
        self.visit: Optional[Visit] = None
        # Audit trail of changes, e.g. reschedules and cancellation
        self._history: List[str] = [f"Booked for {slot}"]

    # ---- read-only properties ---- #
    @property
    def appointment_id(self) -> str:
        return self._appointment_id

    @property
    def patient(self) -> Patient:
        return self._patient

    @property
    def doctor(self) -> Doctor:
        return self._doctor

    @property
    def slot(self) -> TimeSlot:
        return self._slot

    @property
    def status(self) -> AppointmentStatus:
        return self._status

    @property
    def history(self) -> List[str]:
        return list(self._history)

    @property
    def is_active(self) -> bool:
        """BOOKED or RESCHEDULED appointments still occupy the doctor's time."""
        return self._status in (AppointmentStatus.BOOKED, AppointmentStatus.RESCHEDULED)

    # ---- state changes (called by the scheduler / visit) ---- #
    def _reschedule(self, new_slot: TimeSlot) -> None:
        if not self.is_active:
            raise InvalidStateError(
                f"Cannot reschedule a {self._status.value} appointment.")
        self._history.append(f"Rescheduled from {self._slot} to {new_slot}")
        self._slot = new_slot
        self._status = AppointmentStatus.RESCHEDULED

    def _cancel(self, reason: str) -> None:
        if not self.is_active:
            raise InvalidStateError(f"Cannot cancel a {self._status.value} appointment.")
        self._status = AppointmentStatus.CANCELLED
        self.cancellation_reason = reason
        self._history.append(f"Cancelled: {reason}")

    def mark_completed(self) -> None:
        """Called when the visit for this appointment is closed."""
        if not self.is_active:
            raise InvalidStateError(
                f"Cannot complete a {self._status.value} appointment.")
        self._status = AppointmentStatus.COMPLETED
        self._history.append("Completed")

    def __str__(self) -> str:
        return (f"{self._appointment_id}: {self._patient.full_name} with "
                f"{self._doctor.full_name} on {self._slot} [{self._status.value}]")


# ============================================================ #
# Scheduler service
# ============================================================ #
class AppointmentScheduler:
    """Keeps all appointments for a clinic and enforces booking rules."""

    def __init__(self, clinic: Clinic) -> None:
        self._clinic = clinic
```

```
        self._appointments: Dict[str, Appointment] = {}

    # -------------------------- searching ------------------------------ #
    def search_available_slots(self, on_date: date,
                               doctor: Optional[Doctor] = None,
                               speciality: Optional[str] = None,
                               duration_minutes: int = DEFAULT_SLOT_MINUTES,
                               not_before: Optional[datetime] = None
                               ) -> List[AvailableSlot]:
        """List free slots on ``on_date``, filtered by doctor and/or speciality.

        Slots are generated from each doctor's working hours in steps of
        ``duration_minutes``; any slot overlapping an active appointment is
        left out. ``not_before`` hides slots that have already started.
        """
        if duration_minutes <= 0:
            raise ValidationError("Duration must be a positive number of minutes.")
        doctors = [doctor] if doctor else self._clinic.doctors(speciality)
        if doctor and speciality and doctor.speciality.lower() != speciality.lower():
            return []

        results: List[AvailableSlot] = []
        step = timedelta(minutes=duration_minutes)
        for doc in doctors:
            hours = doc.working_hours.hours_for(on_date.weekday())
            if hours is None:
                continue
            cursor = datetime.combine(on_date, hours[0])
            day_end = datetime.combine(on_date, hours[1])
            while cursor + step <= day_end:
                slot = TimeSlot(cursor, cursor + step)
                if (not_before is None or slot.start >= not_before) \
                        and self._doctor_is_free(doc, slot):
                    results.append(AvailableSlot(doc, slot))
                cursor += step
        results.sort(key=lambda s: (s.slot.start, s.doctor.last_name))
        return results

    # -------------------------- booking ------------------------------- #
    def book(self, patient: Patient, doctor: Doctor, start: datetime,
             duration_minutes: int = DEFAULT_SLOT_MINUTES,
             reason: str = "") -> Appointment:
        """Book an appointment after validating availability."""
        slot = TimeSlot.of_minutes(start, duration_minutes)
        self._validate_slot(patient, doctor, slot)
        appointment = Appointment(patient, doctor, slot, reason)
        self._appointments[appointment.appointment_id] = appointment
        return appointment

    def reschedule(self, appointment_id: str, new_start: datetime,
                   duration_minutes: Optional[int] = None) -> Appointment:
        """Move an appointment to a new start time (same doctor)."""
        appointment = self.get(appointment_id)
        minutes = duration_minutes or appointment.slot.duration_minutes
        new_slot = TimeSlot.of_minutes(new_start, minutes)
        self._validate_slot(appointment.patient, appointment.doctor, new_slot,
                            ignore=appointment)
        appointment._reschedule(new_slot)
        return appointment

    def cancel(self, appointment_id: str, reason: str = "Cancelled by patient") -> Appointment:
        """Cancel an appointment; its slot becomes available again."""
        appointment = self.get(appointment_id)
        appointment._cancel(reason)
        return appointment

    # -------------------------- queries ------------------------------- #
    def get(self, appointment_id: str) -> Appointment:
        try:
            return self._appointments[appointment_id]
        except KeyError:
            raise NotFoundError(f"Appointment {appointment_id!r} not found.") from None

    def appointments_for_patient(self, patient: Patient,
                                 upcoming_only: bool = False,
                                 now: Optional[datetime] = None) -> List[Appointment]:
        now = now or datetime.now()
```

```
        items = [a for a in self._appointments.values() if a.patient is patient]
        if upcoming_only:
            items = [a for a in items if a.is_active and a.slot.start >= now]
        return sorted(items, key=lambda a: a.slot.start)

    def appointments_for_doctor(self, doctor: Doctor,
                                on_date: Optional[date] = None) -> List[Appointment]:
        items = [a for a in self._appointments.values() if a.doctor is doctor]
        if on_date:
            items = [a for a in items if a.slot.start.date() == on_date]
        return sorted(items, key=lambda a: a.slot.start)

    # ---------------------------- helpers ------------------------------ #
    def _doctor_is_free(self, doctor: Doctor, slot: TimeSlot,
                        ignore: Optional[Appointment] = None) -> bool:
        return not any(
            a.is_active and a is not ignore and a.doctor is doctor
            and a.slot.overlaps(slot)
            for a in self._appointments.values())

    def _validate_slot(self, patient: Patient, doctor: Doctor, slot: TimeSlot,
                       ignore: Optional[Appointment] = None) -> None:
        if not doctor.is_working(slot.start, slot.end):
            raise SlotUnavailableError(
                f"{doctor.full_name} is not working during {slot}.")
        if not self._doctor_is_free(doctor, slot, ignore):
            raise SlotUnavailableError(f"{doctor.full_name} is already booked at {slot}.")
        clash = any(a.is_active and a is not ignore and a.patient is patient
                    and a.slot.overlaps(slot) for a in self._appointments.values())
        if clash:
            raise SlotUnavailableError(
                f"{patient.full_name} already has an appointment at {slot}.")
```

royal_care/emr.py

Electronic medical records

```
"""
emr.py
======
Electronic Medical Records.

* ``MedicalRecord`` - one per patient; *composition* of ``Visit`` objects.
* ``Visit``         - one per completed appointment; *composition* of
  ``Vitals``, ``Diagnosis``, ``Attachment`` and clinical notes, and
  *association* with ``Prescription`` and ``LabOrder``.
* ``AccessPolicy``  - decides who may read a medical record.
"""

from __future__ import annotations

from datetime import datetime
from typing import TYPE_CHECKING, Dict, List, Optional

from .exceptions import AccessDeniedError, InvalidStateError, NotFoundError, ValidationError
from .utils import IdGenerator

if TYPE_CHECKING:
    from .lab import LabOrder
    from .people import Doctor, Patient, Person, Staff
    from .prescriptions import Prescription
    from .scheduling import Appointment


# ============================================================== #
# Small clinical value objects
# ============================================================== #
class Vitals:
    """Vital signs taken during a visit, with basic range validation."""

    def __init__(self, temperature_c: float, systolic_bp: int, diastolic_bp: int,
                 pulse_bpm: int, respiratory_rate: int, weight_kg: float,
                 height_cm: float, oxygen_saturation: int = 98) -> None:
        self._check(30 <= temperature_c <= 45, "Temperature must be 30-45 °C.")
```

```
        self._check(50 <= systolic_bp <= 260 and 30 <= diastolic_bp <= 160,
                    "Blood pressure is outside the plausible range.")
        self._check(20 <= pulse_bpm <= 250, "Pulse must be 20-250 bpm.")
        self._check(0 < weight_kg < 400 and 30 < height_cm < 260,
                    "Weight/height are outside the plausible range.")
        self._check(50 <= oxygen_saturation <= 100, "SpO2 must be 50-100 %.")
        self.temperature_c = temperature_c
        self.systolic_bp = systolic_bp
        self.diastolic_bp = diastolic_bp
        self.pulse_bpm = pulse_bpm
        self.respiratory_rate = respiratory_rate
        self.weight_kg = weight_kg
        self.height_cm = height_cm
        self.oxygen_saturation = oxygen_saturation
        self.recorded_by: Optional[Staff] = None
        self.recorded_at: Optional[datetime] = None

    @staticmethod
    def _check(condition: bool, message: str) -> None:
        if not condition:
            raise ValidationError(message)

    @property
    def blood_pressure(self) -> str:
        return f"{self.systolic_bp}/{self.diastolic_bp} mmHg"

    @property
    def bmi(self) -> float:
        """Body-mass index rounded to one decimal place."""
        metres = self.height_cm / 100
        return round(self.weight_kg / (metres ** 2), 1)

    def __str__(self) -> str:
        return (f"Temp {self.temperature_c} °C | BP {self.blood_pressure} | "
                f"Pulse {self.pulse_bpm} bpm | RR {self.respiratory_rate}/min | "
                f"SpO2 {self.oxygen_saturation}% | BMI {self.bmi}")


class Diagnosis:
    """A diagnosis with an ICD-10 code."""

    def __init__(self, code: str, description: str, is_primary: bool = False) -> None:
        if not code.strip() or not description.strip():
            raise ValidationError("Diagnosis code and description are required.")
        self.code = code.upper().strip()
        self.description = description.strip()
        self.is_primary = is_primary

    def __str__(self) -> str:
        flag = " (primary)" if self.is_primary else ""
        return f"{self.code} - {self.description}{flag}"


class Attachment:
    """A file attached to a visit, e.g. a scanned referral letter."""

    def __init__(self, file_name: str, content: bytes, file_type: str = "application/pdf",
                 description: str = "") -> None:
        if not content:
            raise ValidationError("An attachment cannot be empty.")
        self.attachment_id = IdGenerator.next_id("ATT")
        self.file_name = file_name
        self.file_type = file_type
        self.description = description
        self._content = content
        self.uploaded_at = datetime.now()

    @property
    def size_bytes(self) -> int:
        return len(self._content)

    @property
    def content(self) -> bytes:
        return self._content

    def __str__(self) -> str:
        return f"{self.file_name} ({self.file_type}, {self.size_bytes} bytes)"
```

```python
class ClinicalNote:
    """A timestamped note written during a visit."""

    def __init__(self, text: str, author: Staff) -> None:
        self.text = text
        self.author = author
        self.written_at = datetime.now()

    def __str__(self) -> str:
        return f"[{self.written_at:%H:%M}] {self.author.full_name}: {self.text}"


# ============================================================================ #
# Visit
# ============================================================================ #
class Visit:
    """Everything recorded during one consultation (one per appointment)."""

    def __init__(self, appointment: Appointment) -> None:
        self._visit_id = IdGenerator.next_id("VIS")
        self._appointment = appointment
        self.started_at = datetime.now()
        self.closed_at: Optional[datetime] = None
        self._notes: List[ClinicalNote] = []
        self._vitals: Optional[Vitals] = None
        self._diagnoses: List[Diagnosis] = []
        self._attachments: List[Attachment] = []
        self._prescriptions: List[Prescription] = []
        self._lab_orders: List[LabOrder] = []

    # ---- links back to the appointment ---- #
    @property
    def visit_id(self) -> str:
        return self._visit_id

    @property
    def appointment(self) -> Appointment:
        return self._appointment

    @property
    def patient(self) -> Patient:
        return self._appointment.patient

    @property
    def doctor(self) -> Doctor:
        return self._appointment.doctor

    @property
    def is_closed(self) -> bool:
        return self.closed_at is not None

    # ---- read-only views of the collections ---- #
    @property
    def notes(self) -> List[ClinicalNote]:
        return list(self._notes)

    @property
    def vitals(self) -> Optional[Vitals]:
        return self._vitals

    @property
    def diagnoses(self) -> List[Diagnosis]:
        return list(self._diagnoses)

    @property
    def attachments(self) -> List[Attachment]:
        return list(self._attachments)

    @property
    def prescriptions(self) -> List[Prescription]:
        return list(self._prescriptions)

    @property
    def lab_orders(self) -> List[LabOrder]:
        return list(self._lab_orders)
```

```python
    # ---- recording clinical data ---- #
    def _ensure_open(self) -> None:
        if self.is_closed:
            raise InvalidStateError(f"Visit {self._visit_id} is closed.")

    def add_note(self, text: str, author: Staff) -> None:
        self._ensure_open()
        self._notes.append(ClinicalNote(text, author))

    def record_vitals(self, vitals: Vitals, recorded_by: Staff) -> None:
        self._ensure_open()
        vitals.recorded_by = recorded_by
        vitals.recorded_at = datetime.now()
        self._vitals = vitals

    def add_diagnosis(self, diagnosis: Diagnosis) -> None:
        self._ensure_open()
        if diagnosis.is_primary and any(d.is_primary for d in self._diagnoses):
            raise ValidationError("A visit can only have one primary diagnosis.")
        self._diagnoses.append(diagnosis)

    def add_attachment(self, attachment: Attachment) -> None:
        self._ensure_open()
        self._attachments.append(attachment)

    def add_prescription(self, prescription: Prescription) -> None:
        self._ensure_open()
        self._prescriptions.append(prescription)

    def add_lab_order(self, order: LabOrder) -> None:
        self._ensure_open()
        self._lab_orders.append(order)

    def close(self) -> None:
        """Finish the visit and mark the appointment as COMPLETED."""
        self._ensure_open()
        self.closed_at = datetime.now()
        self._appointment.mark_completed()

    # ---- reporting ---- #
    def summary(self) -> str:
        """Plain-text visit summary for patients and staff."""
        appt = self._appointment
        lines = [
            "ROYAL CARE CLINIC - VISIT SUMMARY",
            "=" * 48,
            f"Visit ID     : {self._visit_id}",
            f"Appointment  : {appt.appointment_id} ({appt.slot})",
            f"Patient      : {self.patient.full_name} ({self.patient.person_id})",
            f"Doctor       : {self.doctor.full_name} - {self.doctor.speciality}",
            f"Reason       : {appt.reason or 'n/a'}",
            "",
            f"Vitals       : {self._vitals or 'not recorded'}",
            "Diagnoses    :",
        ]
        lines += [f"  - {d}" for d in self._diagnoses] or ["  - none"]
        lines.append("Prescriptions:")
        rx_lines = [f"  - {item}" for rx in self._prescriptions for item in rx.items]
        lines += rx_lines or ["  - none"]
        lines.append("Lab orders   :")
        lines += [f"  - {o.test.name}: {o.status.value}"
                  + (f" -> {o.result.interpretation}" if o.result else "")
                  for o in self._lab_orders] or ["  - none"]
        lines.append("Notes        :")
        lines += [f"  - {n.text}" for n in self._notes] or ["  - none"]
        if self._attachments:
            lines.append("Attachments  :")
            lines += [f"  - {a}" for a in self._attachments]
        return "\n".join(lines) + "\n"


# ============================================================== #
# Medical record and access control
# ============================================================== #
class AccessPolicy:
    """Rules for who may read a patient's medical record.
```

```
    * any member of clinic staff may view visit summaries;
    * a patient may view only their own record;
    * anyone else is refused.
    """

    @staticmethod
    def can_view(requester: Person, record: MedicalRecord) -> bool:
        from .people import Patient, Staff   # local import avoids a cycle
        if isinstance(requester, Staff):
            return True
        if isinstance(requester, Patient):
            return requester is record.patient
        return False


class MedicalRecord:
    """A patient's medical record: the collection of all their visits."""

    def __init__(self, patient: Patient) -> None:
        self._record_id = IdGenerator.next_id("EMR")
        self._patient = patient
        self._visits: Dict[str, Visit] = {}

    @property
    def record_id(self) -> str:
        return self._record_id

    @property
    def patient(self) -> Patient:
        return self._patient

    def add_visit(self, visit: Visit) -> None:
        if visit.patient is not self._patient:
            raise ValidationError("Visit belongs to a different patient.")
        self._visits[visit.visit_id] = visit

    def _authorise(self, requester: Person) -> None:
        if not AccessPolicy.can_view(requester, self):
            raise AccessDeniedError(
                f"{requester.full_name} is not allowed to view this medical record.")

    def get_visit(self, visit_id: str, requester: Person) -> Visit:
        """Return a visit after checking the requester's access rights."""
        self._authorise(requester)
        try:
            return self._visits[visit_id]
        except KeyError:
            raise NotFoundError(f"Visit {visit_id!r} not found in this record.") from None

    def visits(self, requester: Person) -> List[Visit]:
        """All visits in date order, after an access check."""
        self._authorise(requester)
        return sorted(self._visits.values(), key=lambda v: v.appointment.slot.start)

    def __len__(self) -> int:
        return len(self._visits)
```

royal_care/prescriptions.py

Prescriptions and medication

```
"""
prescriptions.py
================
Prescriptions and medication management.

* ``Medication``       - an item in the pharmacy formulary with a unit price.
* ``PrescriptionItem`` - drug name, dosage and directions for one medication.
* ``Prescription``     - *composition* of ``PrescriptionItem`` objects; issued
  by a Doctor for a Visit and sent to the Pharmacy.
"""

from __future__ import annotations
```

```python
from datetime import datetime
from decimal import Decimal
from typing import TYPE_CHECKING, List, Optional

from .enums import PrescriptionStatus
from .exceptions import InvalidStateError, ValidationError
from .utils import IdGenerator, to_money

if TYPE_CHECKING:
    from .clinic import Pharmacy
    from .emr import Visit
    from .people import Doctor, Patient, Pharmacist


class Medication:
    """A medication available from the pharmacy."""

    def __init__(self, name: str, strength: str, form: str, unit_price) -> None:
        if not name.strip():
            raise ValidationError("Medication name is required.")
        self.name = name.strip()
        self.strength = strength
        self.form = form
        self.unit_price = to_money(unit_price)
        if self.unit_price < 0:
            raise ValidationError("Unit price cannot be negative.")

    def __str__(self) -> str:
        return f"{self.name} {self.strength} {self.form}"


class PrescriptionItem:
    """One line of a prescription: drug, dosage, directions and quantity."""

    def __init__(self, medication: Medication, dosage: str, directions: str,
                 quantity: int, duration_days: int = 0) -> None:
        if quantity <= 0:
            raise ValidationError("Quantity must be at least 1.")
        if not dosage.strip() or not directions.strip():
            raise ValidationError("Dosage and directions are required.")
        self.medication = medication
        self.dosage = dosage
        self.directions = directions
        self.quantity = quantity
        self.duration_days = duration_days

    @property
    def drug_name(self) -> str:
        return self.medication.name

    @property
    def line_total(self) -> Decimal:
        return to_money(self.medication.unit_price * self.quantity)

    def __str__(self) -> str:
        return (f"{self.medication} - {self.dosage}, {self.directions} "
                f"(qty {self.quantity})")


class Prescription:
    """A prescription issued during a visit."""

    def __init__(self, visit: Visit, doctor: Doctor,
                 items: List[PrescriptionItem]) -> None:
        if not items:
            raise ValidationError("A prescription needs at least one item.")
        self._prescription_id = IdGenerator.next_id("RX")
        self._visit = visit
        self._doctor = doctor
        self._items: List[PrescriptionItem] = list(items)   # composition
        self._status = PrescriptionStatus.DRAFT
        self.issued_at = datetime.now()
        self.sent_at: Optional[datetime] = None
        self.pharmacy: Optional[Pharmacy] = None
        self.dispensed_by: Optional[Pharmacist] = None
```

```
    @property
    def prescription_id(self) -> str:
        return self._prescription_id

    @property
    def visit(self) -> Visit:
        return self._visit

    @property
    def doctor(self) -> Doctor:
        return self._doctor

    @property
    def patient(self) -> Patient:
        return self._visit.patient

    @property
    def items(self) -> List[PrescriptionItem]:
        return list(self._items)

    @property
    def status(self) -> PrescriptionStatus:
        return self._status

    @property
    def total_cost(self) -> Decimal:
        return to_money(sum((i.line_total for i in self._items), Decimal("0")))

    def add_item(self, item: PrescriptionItem) -> None:
        """Items can only be added while the prescription is still a draft."""
        if self._status is not PrescriptionStatus.DRAFT:
            raise InvalidStateError("Cannot change a prescription after it is sent.")
        self._items.append(item)

    def send_to_pharmacy(self, pharmacy: Pharmacy) -> None:
        """Send the prescription to the pharmacy (DRAFT -> SENT_TO_PHARMACY)."""
        if self._status is not PrescriptionStatus.DRAFT:
            raise InvalidStateError(f"Prescription already {self._status.value}.")
        self._status = PrescriptionStatus.SENT_TO_PHARMACY
        self.sent_at = datetime.now()
        self.pharmacy = pharmacy
        pharmacy.receive_prescription(self)

    def mark_dispensed(self, pharmacist: Pharmacist) -> None:
        """Record that medication was handed to the patient."""
        if self._status is not PrescriptionStatus.SENT_TO_PHARMACY:
            raise InvalidStateError("Only prescriptions sent to the pharmacy can be dispensed.")
        self._status = PrescriptionStatus.DISPENSED
        self.dispensed_by = pharmacist

    def __str__(self) -> str:
        return (f"{self._prescription_id} for {self.patient.full_name} by "
                f"{self._doctor.full_name} [{self._status.value}]")
```

royal_care/lab.py

Laboratory orders and results

```
"""
lab.py
======
Laboratory orders and results.

* ``LabTest``   - a test in the laboratory catalogue (code, name, price).
* ``LabOrder``  - a test ordered by a doctor for a specific visit; it moves
  through ORDERED -> SAMPLE_COLLECTED -> COMPLETED.
* ``LabResult`` - *composition*: the result belongs to exactly one order.

A result is only accepted if the patient and visit supplied by the lab
technician match the order, so results cannot be filed against the wrong
patient.
"""

from __future__ import annotations
```

```python
from datetime import datetime
from typing import TYPE_CHECKING, Dict, Optional

from .enums import LabOrderStatus
from .exceptions import InvalidStateError, ValidationError
from .utils import IdGenerator, to_money

if TYPE_CHECKING:
    from .emr import Visit
    from .people import Doctor, Patient, Staff


class LabTest:
    """A laboratory test offered by the clinic."""

    def __init__(self, code: str, name: str, specimen: str, price,
                 turnaround_hours: int = 24) -> None:
        self.code = code.upper()
        self.name = name
        self.specimen = specimen
        self.price = to_money(price)
        self.turnaround_hours = turnaround_hours

    def __str__(self) -> str:
        return f"{self.code} - {self.name} ({self.specimen})"


class LabResult:
    """The outcome of a lab test."""

    def __init__(self, values: Dict[str, str], interpretation: str,
                 is_abnormal: bool = False, recorded_by: Optional[Staff] = None) -> None:
        if not values:
            raise ValidationError("A lab result must contain at least one value.")
        self.values = dict(values)
        self.interpretation = interpretation
        self.is_abnormal = is_abnormal
        self.recorded_by = recorded_by
        self.recorded_at = datetime.now()

    def __str__(self) -> str:
        readings = ", ".join(f"{k}: {v}" for k, v in self.values.items())
        flag = "ABNORMAL" if self.is_abnormal else "normal"
        return f"{readings} -> {self.interpretation} ({flag})"


class LabOrder:
    """A lab test ordered for a visit."""

    def __init__(self, visit: Visit, test: LabTest, ordered_by: Doctor,
                 notes: str = "") -> None:
        self._order_id = IdGenerator.next_id("LBO")
        self._visit = visit
        self._test = test
        self._ordered_by = ordered_by
        self.notes = notes
        self._status = LabOrderStatus.ORDERED
        self.ordered_at = datetime.now()
        self.sample_collected_at: Optional[datetime] = None
        self._result: Optional[LabResult] = None

    # ---- read-only properties ---- #
    @property
    def order_id(self) -> str:
        return self._order_id

    @property
    def visit(self) -> Visit:
        return self._visit

    @property
    def patient(self) -> Patient:
        return self._visit.patient

    @property
    def test(self) -> LabTest:
```

```
        return self._test

    @property
    def ordered_by(self) -> Doctor:
        return self._ordered_by

    @property
    def status(self) -> LabOrderStatus:
        return self._status

    @property
    def result(self) -> Optional[LabResult]:
        return self._result

    @property
    def is_complete(self) -> bool:
        return self._status is LabOrderStatus.COMPLETED

    # ---- lifecycle ---- #
    def collect_sample(self) -> None:
        if self._status is not LabOrderStatus.ORDERED:
            raise InvalidStateError(f"Cannot collect sample when order is {self._status.value}.")
        self._status = LabOrderStatus.SAMPLE_COLLECTED
        self.sample_collected_at = datetime.now()

    def record_result(self, result: LabResult, patient: Patient, visit_id: str) -> None:
        """Store the result after confirming it is for the right patient and visit."""
        if patient is not self.patient:
            raise ValidationError(
                f"Result is for {patient.full_name} but order {self._order_id} "
                f"belongs to {self.patient.full_name}.")
        if visit_id != self._visit.visit_id:
            raise ValidationError(
                f"Visit {visit_id} does not match order visit {self._visit.visit_id}.")
        if self._status in (LabOrderStatus.COMPLETED, LabOrderStatus.CANCELLED):
            raise InvalidStateError(f"Order {self._order_id} is already {self._status.value}.")
        if self._status is LabOrderStatus.ORDERED:      # sample taken implicitly
            self.collect_sample()
        self._result = result
        self._status = LabOrderStatus.COMPLETED

    def cancel(self) -> None:
        if self._status is LabOrderStatus.COMPLETED:
            raise InvalidStateError("A completed lab order cannot be cancelled.")
        self._status = LabOrderStatus.CANCELLED

    def __str__(self) -> str:
        return f"{self._order_id}: {self._test.name} for {self.patient.full_name} [{self._status.value}]"
```

`royal_care/billing.py`

Billing, insurance and payments

```
"""
billing.py
==========
Billing, insurance and payments.

* ``InsurancePolicy`` - coverage percentage, flat co-pay and annual limit.
* ``InvoiceLineItem`` - one charge on an invoice.
* ``Invoice``         - *composition* of line items and *aggregation* of
  payments; generated per visit.
* ``Payment`` (abstract) with subclasses ``CardPayment``,
  ``MobileWalletPayment`` and ``InsuranceClaimPayment`` (inheritance and
  polymorphism: each subclass authorises the payment in its own way).
* ``BillingService``  - builds an invoice from everything done in a visit.

Insurance / co-pay rule used
----------------------------
1. ``eligible``  = sum of charges in categories the policy covers.
2. ``co_pay``    = the policy's flat co-pay (never more than ``eligible``).
3. ``insurer``   = (eligible - co_pay) x coverage %, capped at the policy's
   remaining annual limit.
4. ``patient``   = invoice total - insurer share.
```

```
"""

from __future__ import annotations

from abc import ABC, abstractmethod
from datetime import date, datetime
from decimal import Decimal
from typing import TYPE_CHECKING, Iterable, List, Optional, Set

from .enums import ChargeCategory, InvoiceStatus, LabOrderStatus, PaymentStatus
from .exceptions import InvalidStateError, PaymentError, ValidationError
from .utils import IdGenerator, format_money, to_money

if TYPE_CHECKING:
    from .emr import Visit
    from .people import Patient

ZERO = Decimal("0.00")


# ============================================================== #
# Insurance
# ============================================================== #
class InsurancePolicy:
    """A patient's health insurance policy."""

    def __init__(self, provider: str, policy_number: str, coverage_percent: float,
                 co_pay, annual_limit, valid_until: date,
                 covered_categories: Optional[Iterable[ChargeCategory]] = None) -> None:
        if not 0 <= coverage_percent <= 100:
            raise ValidationError("Coverage percent must be between 0 and 100.")
        self.provider = provider
        self._policy_number = policy_number
        self.coverage_percent = Decimal(str(coverage_percent))
        self.co_pay = to_money(co_pay)
        self.annual_limit = to_money(annual_limit)
        self.valid_until = valid_until
        self.covered_categories: Set[ChargeCategory] = set(
            covered_categories or list(ChargeCategory))
        self._claimed_to_date = ZERO

    @property
    def policy_number(self) -> str:
        return self._policy_number

    @property
    def remaining_limit(self) -> Decimal:
        return to_money(self.annual_limit - self._claimed_to_date)

    def is_valid(self, on: Optional[date] = None) -> bool:
        return (on or date.today()) <= self.valid_until

    def covers(self, category: ChargeCategory) -> bool:
        return category in self.covered_categories

    def record_claim(self, amount: Decimal) -> None:
        """Reduce the remaining annual limit after a paid claim."""
        if amount > self.remaining_limit:
            raise PaymentError("Claim exceeds the remaining annual limit.")
        self._claimed_to_date = to_money(self._claimed_to_date + amount)

    def __str__(self) -> str:
        return (f"{self.provider} #{self._policy_number} - {self.coverage_percent}% "
                f"cover, co-pay {format_money(self.co_pay)}")


# ============================================================== #
# Invoice
# ============================================================== #
class InvoiceLineItem:
    """A single charge on an invoice."""

    def __init__(self, description: str, category: ChargeCategory,
                 quantity: int, unit_price) -> None:
        if quantity <= 0:
            raise ValidationError("Quantity must be positive.")
        self.description = description
```

```
        self.category = category
        self.quantity = quantity
        self.unit_price = to_money(unit_price)

    @property
    def total(self) -> Decimal:
        return to_money(self.unit_price * self.quantity)


class Invoice:
    """An invoice for one visit."""

    def __init__(self, visit: Visit) -> None:
        self._invoice_id = IdGenerator.next_id("INV")
        self._visit = visit
        self.issued_at = datetime.now()
        self._items: List[InvoiceLineItem] = []      # composition
        self._payments: List[Payment] = []           # aggregation
        self.policy: Optional[InsurancePolicy] = None
        self._insurer_share = ZERO
        self._co_pay = ZERO

    # ---- basic properties ---- #
    @property
    def invoice_id(self) -> str:
        return self._invoice_id

    @property
    def visit(self) -> Visit:
        return self._visit

    @property
    def patient(self) -> Patient:
        return self._visit.patient

    @property
    def items(self) -> List[InvoiceLineItem]:
        return list(self._items)

    @property
    def payments(self) -> List[Payment]:
        return list(self._payments)

    def add_item(self, item: InvoiceLineItem) -> None:
        if self._payments:
            raise InvalidStateError("Cannot add charges after payments have been taken.")
        self._items.append(item)
        if self.policy:                       # keep insurance figures up to date
            self.apply_insurance(self.policy)

    # ---- totals ---- #
    @property
    def total(self) -> Decimal:
        return to_money(sum((i.total for i in self._items), ZERO))

    def subtotal(self, category: ChargeCategory) -> Decimal:
        return to_money(sum((i.total for i in self._items if i.category is category), ZERO))

    @property
    def insurer_share(self) -> Decimal:
        return self._insurer_share

    @property
    def co_pay(self) -> Decimal:
        return self._co_pay

    @property
    def patient_share(self) -> Decimal:
        return to_money(self.total - self._insurer_share)

    def _paid(self, insurer: bool) -> Decimal:
        return to_money(sum((p.amount for p in self._payments
                             if p.status is PaymentStatus.SUCCESSFUL
                             and isinstance(p, InsuranceClaimPayment) == insurer), ZERO))

    @property
    def patient_outstanding(self) -> Decimal:
```

```
        return to_money(self.patient_share - self._paid(insurer=False))

    @property
    def insurer_outstanding(self) -> Decimal:
        return to_money(self._insurer_share - self._paid(insurer=True))

    @property
    def balance_due(self) -> Decimal:
        return to_money(self.patient_outstanding + self.insurer_outstanding)

    @property
    def status(self) -> InvoiceStatus:
        if self.balance_due == ZERO and self.total > ZERO:
            return InvoiceStatus.PAID
        if any(p.status is PaymentStatus.SUCCESSFUL for p in self._payments):
            return InvoiceStatus.PARTIALLY_PAID
        return InvoiceStatus.UNPAID

    # ---- insurance ---- #
    def apply_insurance(self, policy: InsurancePolicy,
                        on: Optional[date] = None) -> Decimal:
        """Calculate the insurer's share and co-pay; returns the insurer share."""
        if self._payments:
            raise InvalidStateError("Insurance must be applied before payments are taken.")
        if not policy.is_valid(on):
            raise ValidationError(f"Insurance policy {policy.policy_number} has expired.")
        eligible = to_money(sum((i.total for i in self._items
                                 if policy.covers(i.category)), ZERO))
        self._co_pay = min(policy.co_pay, eligible)
        claimable = to_money((eligible - self._co_pay) * policy.coverage_percent / 100)
        self._insurer_share = min(claimable, policy.remaining_limit)
        self.policy = policy
        return self._insurer_share

    # ---- payments ---- #
    def _register_payment(self, payment: Payment) -> None:
        """Called by ``Payment.process`` after a successful authorisation."""
        self._payments.append(payment)

    def __str__(self) -> str:
        return (f"{self._invoice_id} ({self.patient.full_name}) total "
                f"{format_money(self.total)} [{self.status.value}]")


# ============================================================================ #
# Payments (abstract base + concrete methods)
# ============================================================================ #
class Payment(ABC):
    """Abstract payment. Subclasses implement :meth:`_authorise`.

    :meth:`process` is a *template method*: the shared steps (validation,
    recording on the invoice) are written once here, while the
    method-specific authorisation step is supplied by each subclass.
    """

    def __init__(self, amount) -> None:
        self._payment_id = IdGenerator.next_id("PAY")
        self._amount = to_money(amount)
        if self._amount <= ZERO:
            raise ValidationError("Payment amount must be positive.")
        self._status = PaymentStatus.PENDING
        self.paid_at: Optional[datetime] = None
        self.reference: Optional[str] = None

    @property
    def payment_id(self) -> str:
        return self._payment_id

    @property
    def amount(self) -> Decimal:
        return self._amount

    @property
    def status(self) -> PaymentStatus:
        return self._status

    @property
```

```python
    @abstractmethod
    def method_name(self) -> str:
        """Display name of the payment method."""

    @abstractmethod
    def _authorise(self, invoice: Invoice) -> str:
        """Authorise the payment and return a reference; raise PaymentError if refused."""

    def _outstanding(self, invoice: Invoice) -> Decimal:
        """The part of the invoice this payment type is allowed to settle."""
        return invoice.patient_outstanding

    def process(self, invoice: Invoice) -> "Payment":
        """Validate, authorise and record the payment against ``invoice``."""
        if self._status is not PaymentStatus.PENDING:
            raise InvalidStateError("This payment has already been processed.")
        outstanding = self._outstanding(invoice)
        if self._amount > outstanding:
            self._status = PaymentStatus.FAILED
            raise PaymentError(
                f"{self.method_name} payment of {format_money(self._amount)} exceeds "
                f"the amount due ({format_money(outstanding)}).")
        try:
            self.reference = self._authorise(invoice)
        except PaymentError:
            self._status = PaymentStatus.FAILED
            raise
        self._status = PaymentStatus.SUCCESSFUL
        self.paid_at = datetime.now()
        invoice._register_payment(self)
        return self

    def __str__(self) -> str:
        return (f"{self._payment_id} {self.method_name} {format_money(self._amount)} "
                f"[{self._status.value}]")


class CardPayment(Payment):
    """Payment by debit/credit card (number checked with the Luhn algorithm)."""

    def __init__(self, amount, card_number: str, card_holder: str, expiry: str) -> None:
        super().__init__(amount)
        digits = card_number.replace(" ", "")
        if not digits.isdigit() or not 12 <= len(digits) <= 19:
            raise ValidationError("Card number must contain 12-19 digits.")
        self._card_number = digits          # kept private, only masked form is shown
        self.card_holder = card_holder
        self.expiry = expiry                # "MM/YY"

    @property
    def method_name(self) -> str:
        return "Card"

    @property
    def masked_number(self) -> str:
        return "**** **** **** " + self._card_number[-4:]

    @staticmethod
    def luhn_valid(number: str) -> bool:
        """Standard Luhn checksum used by card networks."""
        total = 0
        for i, ch in enumerate(reversed(number)):
            d = int(ch)
            if i % 2 == 1:
                d = d * 2 - 9 if d > 4 else d * 2
            total += d
        return total % 10 == 0

    def _is_expired(self) -> bool:
        month, year = self.expiry.split("/")
        expiry_key = (2000 + int(year), int(month))
        today = date.today()
        return expiry_key < (today.year, today.month)

    def _authorise(self, invoice: Invoice) -> str:
        if not self.luhn_valid(self._card_number):
            raise PaymentError("Card declined: invalid card number.")
```

```
        if self._is_expired():
            raise PaymentError("Card declined: card has expired.")
        return f"CARD-AUTH-{self._card_number[-4:]}-{self.payment_id}"


class MobileWalletPayment(Payment):
    """Payment through a mobile wallet (e.g. Apple Pay, M-Pesa)."""

    def __init__(self, amount, provider: str, phone_number: str) -> None:
        super().__init__(amount)
        if not phone_number.replace("+", "").replace(" ", "").isdigit():
            raise ValidationError("Wallet phone number must contain digits only.")
        self.provider = provider
        self.phone_number = phone_number

    @property
    def method_name(self) -> str:
        return f"Mobile Wallet ({self.provider})"

    def _authorise(self, invoice: Invoice) -> str:
        return f"WALLET-{self.provider.upper().replace(' ', '')}-{self.payment_id}"


class InsuranceClaimPayment(Payment):
    """Settlement of the insurer's share through an insurance claim."""

    def __init__(self, amount, policy: InsurancePolicy) -> None:
        super().__init__(amount)
        self.policy = policy
        self.claim_number = IdGenerator.next_id("CLM")

    @property
    def method_name(self) -> str:
        return f"Insurance Claim ({self.policy.provider})"

    def _outstanding(self, invoice: Invoice) -> Decimal:
        return invoice.insurer_outstanding

    def _authorise(self, invoice: Invoice) -> str:
        if invoice.policy is not self.policy:
            raise PaymentError("Insurance has not been applied to this invoice with this policy.")
        self.policy.record_claim(self.amount)
        return self.claim_number


# ============================================================================ #
# Billing service
# ============================================================================ #
class BillingService:
    """Generates invoices for visits and keeps them for lookup."""

    def __init__(self) -> None:
        self._invoices: dict[str, Invoice] = {}

    def generate_invoice(self, visit: Visit,
                         other_services: Optional[List[InvoiceLineItem]] = None,
                         apply_patient_insurance: bool = True) -> Invoice:
        """Create an invoice containing consultation, lab, medication and other charges.

        If the patient has a valid insurance policy it is applied automatically.
        """
        if any(inv.visit is visit for inv in self._invoices.values()):
            raise InvalidStateError(f"Visit {visit.visit_id} has already been invoiced.")
        invoice = Invoice(visit)
        doctor = visit.doctor
        invoice.add_item(InvoiceLineItem(
            f"Consultation - {doctor.full_name} ({doctor.speciality})",
            ChargeCategory.CONSULTATION, 1, doctor.consultation_fee))
        for order in visit.lab_orders:
            if order.status is not LabOrderStatus.CANCELLED:
                invoice.add_item(InvoiceLineItem(
                    f"Lab test - {order.test.name}", ChargeCategory.LAB, 1, order.test.price))
        for prescription in visit.prescriptions:
            for item in prescription.items:
                invoice.add_item(InvoiceLineItem(
                    f"Medication - {item.medication}", ChargeCategory.MEDICATION,
                    item.quantity, item.medication.unit_price))
```

```
        for service in other_services or []:
            invoice.add_item(service)

        policy = visit.patient.insurance
        if apply_patient_insurance and policy is not None and policy.is_valid():
            invoice.apply_insurance(policy)
        self._invoices[invoice.invoice_id] = invoice
        return invoice

    def get_invoice(self, invoice_id: str) -> Invoice:
        return self._invoices[invoice_id]

    @property
    def invoices(self) -> List[Invoice]:
        return list(self._invoices.values())
```

royal_care/formatting.py

Console output helpers

```
"""
formatting.py
=============
Helpers that turn objects into neat console output (headings, key/value
blocks and aligned tables). Keeping presentation here keeps the domain
classes free of printing code.
"""

from typing import Iterable, List, Sequence, Tuple

WIDTH = 78


def heading(title: str, char: str = "=") -> str:
    """A centred title between two rules."""
    rule = char * WIDTH
    return f"\n{rule}\n{title.upper().center(WIDTH)}\n{rule}"


def subheading(title: str) -> str:
    return f"\n--- {title} " + "-" * max(0, WIDTH - len(title) - 5)


def key_values(pairs: Iterable[Tuple[str, object]], indent: int = 2) -> str:
    """Align ``label : value`` pairs."""
    pairs = list(pairs)
    width = max((len(k) for k, _ in pairs), default=0)
    pad = " " * indent
    return "\n".join(f"{pad}{k.ljust(width)} : {v}" for k, v in pairs)


def table(headers: Sequence[str], rows: Sequence[Sequence[object]],
          align_right: Sequence[int] = ()) -> str:
    """Render rows as a simple text table with a header rule.

    ``align_right`` lists column indexes to right-align (useful for money).
    """
    text_rows: List[List[str]] = [[str(c) for c in row] for row in rows]
    widths = [len(h) for h in headers]
    for row in text_rows:
        for i, cell in enumerate(row):
            widths[i] = max(widths[i], len(cell))

    def fmt(cells: Sequence[str]) -> str:
        out = []
        for i, cell in enumerate(cells):
            out.append(cell.rjust(widths[i]) if i in align_right else cell.ljust(widths[i]))
        return "  " + " | ".join(out)

    rule = "  " + "-+-".join("-" * w for w in widths)
    lines = [fmt(list(headers)), rule] + [fmt(r) for r in text_rows]
    return "\n".join(lines)
```

royal_care/sample_data.py

Sample clinic set-up

```
"""
sample_data.py
==============
Builds a fully configured Royal Care Clinic (departments, staff, lab test
catalogue and pharmacy formulary). Shared by ``main.py`` and the tests so
both work with the same, realistic set-up.
"""

from datetime import date

from .clinic import Address, Clinic, ContactInfo, Department, Laboratory, Pharmacy
from .hours import WeeklyHours
from .lab import LabTest
from .people import Doctor, LabTechnician, Nurse, Pharmacist
from .prescriptions import Medication


def build_royal_care_clinic() -> Clinic:
    """Create and return the Royal Care Clinic with sample departments and staff."""
    clinic = Clinic(
        name="Royal Care Clinic",
        address=Address("14 Harbour View Road", "Kingston", "Jamaica", "KN10"),
        contact=ContactInfo(["+1 876 555 0100", "+1 876 555 0101"],
                            "info@royalcareclinic.example",
                            "www.royalcareclinic.example"),
    )

    # ---------------------------- departments ---------------------------- #
    saturday_hours = WeeklyHours.weekdays("08:00", "18:00")
    saturday_hours.set_day(5, "09:00", "13:00")
    clinic.add_department(Department("General Practice", "Primary care consultations",
                                     "Ground floor, Wing A", "ext. 201", saturday_hours))
    clinic.add_department(Department("Cardiology", "Heart and circulation",
                                     "First floor, Wing B", "ext. 305"))
    clinic.add_department(Department("Paediatrics", "Care for children and adolescents",
                                     "Ground floor, Wing C", "ext. 220"))
    clinic.add_department(Department("Radiology", "X-ray and ultrasound imaging",
                                     "Basement", "ext. 110"))
    lab = clinic.add_department(Laboratory(description="Pathology and blood tests",
                                           location="Basement", phone="ext. 115"))
    pharmacy = clinic.add_department(Pharmacy(description="Outpatient dispensary",
                                              location="Ground floor, Lobby",
                                              phone="ext. 100"))

    # ------------------------------ doctors ------------------------------ #
    clinic.add_staff(Doctor("Michael", "Brown", date(1978, 4, 12), "+1 876 555 0110",
                            "m.brown@royalcareclinic.example", "General Practice",
                            "MD-GP-1042", 80, ["MBBS", "DFM"],
                            WeeklyHours.weekdays("09:00", "12:00")),
                     "General Practice")
    clinic.add_staff(Doctor("Ngozi", "Okafor", date(1981, 9, 3), "+1 876 555 0111",
                            "n.okafor@royalcareclinic.example", "Cardiology",
                            "MD-CA-2210", 150, ["MBBS", "MRCP", "FACC"],
                            WeeklyHours.weekdays("13:00", "16:00")),
                     "Cardiology")
    clinic.add_staff(Doctor("Ravi", "Patel", date(1985, 1, 22), "+1 876 555 0112",
                            "r.patel@royalcareclinic.example", "Paediatrics",
                            "MD-PD-3307", 95, ["MBBS", "DCH"],
                            WeeklyHours.weekdays("09:00", "15:00")),
                     "Paediatrics")

    # --------------------------- other staff --------------------------- #
    clinic.add_staff(Nurse("Grace", "Thompson", date(1990, 6, 14), "+1 876 555 0120",
                           "g.thompson@royalcareclinic.example",
                           qualifications=["RN", "BSc Nursing"]), "General Practice")
    clinic.add_staff(LabTechnician("Daniel", "Reid", date(1992, 11, 30), "+1 876 555 0130",
                                   "d.reid@royalcareclinic.example",
                                   qualifications=["BSc Medical Technology"]), "Laboratory")
    clinic.add_staff(Pharmacist("Sophia", "Campbell", date(1988, 2, 8), "+1 876 555 0140",
                                "s.campbell@royalcareclinic.example",
                                qualifications=["BPharm"]), "Pharmacy")
```

```
    # ------------------------ catalogues ------------------------------ #
    for test in (LabTest("FBC", "Full Blood Count", "Blood", 25),
                 LabTest("LIPID", "Lipid Panel", "Blood", 40),
                 LabTest("HBA1C", "HbA1c (Glycated Haemoglobin)", "Blood", 35),
                 LabTest("URINE", "Urinalysis", "Urine", 15),
                 LabTest("STREP", "Rapid Strep A Test", "Throat swab", 18, 1)):
        lab.add_test(test)

    for med in (Medication("Amoxicillin", "500 mg", "capsule", 0.80),
                Medication("Paracetamol", "500 mg", "tablet", 0.10),
                Medication("Ibuprofen", "400 mg", "tablet", 0.15),
                Medication("Atorvastatin", "20 mg", "tablet", 0.50),
                Medication("Penicillin V", "250 mg", "tablet", 0.30)):
        pharmacy.add_medication(med)

    return clinic
```

main.py

Demonstration program

```
"""
main.py
=======
End-to-end demonstration of the Royal Care Clinic Management System.

Scenario (from the assignment brief)
------------------------------------
1. Create patient Aliyah and doctor Dr. Brown (General Practitioner).
2. Aliyah searches for and books a 30-minute appointment next Monday.
3. The doctor conducts the visit, records vitals and a diagnosis, orders a
   lab test and issues a prescription (sent to the pharmacy).
4. Lab results are uploaded, the invoice is generated with insurance
   applied, and payment is completed.

The script also shows rescheduling, cancellation, access control and the
patient downloading a visit summary.

Run with:  python main.py
"""

from datetime import date, datetime, time, timedelta

from royal_care import (AccessDeniedError, Attachment, ChargeCategory, CardPayment,
                        Diagnosis, EmergencyContact, InsuranceClaimPayment,
                        InsurancePolicy, InvoiceLineItem, LabTechnician,
                        MobileWalletPayment, Nurse, Pharmacist, PrescriptionItem,
                        ValidationError, Vitals)
from royal_care.clinic import Address
from royal_care.formatting import heading, key_values, subheading, table
from royal_care.sample_data import build_royal_care_clinic
from royal_care.utils import format_money, next_weekday, to_money


def main() -> None:
    clinic = build_royal_care_clinic()

    # ================================================================ #
    # 1. Clinic information
    # ================================================================ #
    print(heading("Royal Care Clinic Management System"))
    info = clinic.get_info()
    print(subheading("Clinic profile"))
    print(key_values([("Name", info["name"]), ("Address", info["address"]),
                      ("Phone", ", ".join(info["phone_numbers"])),
                      ("Email", info["email"]), ("Website", info["website"])]))

    print(subheading("Departments"))
    print(table(["Department", "Type", "Location", "Mon hours", "Sat hours", "Staff"],
                [[d.name, type(d).__name__, d.location,
                  d.operating_hours.describe()[0].split(": ")[1],
                  d.operating_hours.describe()[5].split(": ")[1], len(d.staff)]
                 for d in clinic.departments]))
```

```
clinic.update_department("Radiology", phone="ext. 111")
print("\n  Updated Radiology phone ->", clinic.get_department("Radiology").phone)

print(subheading("Doctors"))
print(table(["ID", "Name", "Speciality", "Qualifications", "Fee"],
            [[d.person_id, d.full_name, d.speciality, ", ".join(d.qualifications),
              format_money(d.consultation_fee)] for d in clinic.doctors()],
            align_right=[4]))

# ================================================================== #
# 2. Patient account creation
# ================================================================== #
print(heading("Patient registration"))
insurance = InsurancePolicy("Sagicor Health", "SH-778812", coverage_percent=80,
                            co_pay=20, annual_limit=5000,
                            valid_until=date.today() + timedelta(days=365))
aliyah = clinic.register_patient(
    "Aliyah", "Johnson", date(1996, 7, 19), "+1 876 555 0199",
    "aliyah.johnson@example.com",
    Address("27 Palm Avenue", "Kingston", "Jamaica", "KN5"),
    username="aliyahj", password="Secure#2026",
    emergency_contact=EmergencyContact("Marcus Johnson", "Brother", "+1 876 555 0177"),
    insurance=insurance, allergies=["Penicillin V"],
    medical_history_summary="Mild asthma (childhood); no surgeries.")
logged_in = clinic.login("aliyahj", "Secure#2026")
print(key_values([("Patient ID", aliyah.person_id), ("Name", aliyah.full_name),
                  ("Age", aliyah.age()), ("Email", aliyah.email),
                  ("Emergency contact", aliyah.emergency_contact),
                  ("Insurance", aliyah.insurance),
                  ("Allergies", ", ".join(aliyah.allergies)),
                  ("History", aliyah.medical_history_summary),
                  ("Login check", "OK" if logged_in is aliyah else "FAILED")]))
aliyah.update_profile(phone="+1 876 555 0200")
print(f"\n  Profile updated: new phone {aliyah.phone}")

# ================================================================== #
# 3. Search and book an appointment
# ================================================================== #
print(heading("Appointment scheduling"))
dr_brown = clinic.doctors("General Practice")[0]
monday = next_weekday(date.today(), 0)
slots = clinic.scheduler.search_available_slots(monday, speciality="General Practice")
print(subheading(f"Available GP slots on {monday:%A %d %B %Y}"))
print(table(["#", "Doctor", "Slot"],
            [[i + 1, s.doctor.full_name, str(s.slot)] for i, s in enumerate(slots)]))

appointment = clinic.scheduler.book(aliyah, dr_brown, datetime.combine(monday, time(9, 30)),
                                    30, reason="Sore throat and fever for 3 days")
print(f"\n  Booked   : {appointment}")
clinic.scheduler.reschedule(appointment.appointment_id,
                            datetime.combine(monday, time(10, 0)))
print(f"  Moved    : {appointment}")

# A second appointment that the patient later cancels
dr_okafor = clinic.doctors("Cardiology")[0]
follow_up = clinic.scheduler.book(aliyah, dr_okafor,
                                  datetime.combine(monday + timedelta(days=2), time(14, 0)),
                                  reason="Palpitations review")
clinic.scheduler.cancel(follow_up.appointment_id, "Symptoms resolved")
print(f"  Cancelled: {follow_up}")

print(subheading("Aliyah's upcoming appointments (patient portal)"))
for appt in aliyah.upcoming_appointments(clinic.scheduler):
    print(f"  {appt}")

# ================================================================== #
# 4. The visit (EMR)
# ================================================================== #
print(heading("Visit and electronic medical record"))
nurse = next(s for s in clinic.get_department("General Practice").staff
             if isinstance(s, Nurse))
lab = clinic.laboratory()
pharmacy = clinic.pharmacy()

visit = dr_brown.start_visit(appointment)
nurse.record_vitals(visit, Vitals(38.4, 122, 78, 96, 18, 64.5, 167, 97))
```

```
    visit.add_note("Erythematous pharynx with tonsillar exudate; no cough.", dr_brown)
    visit.add_diagnosis(Diagnosis("J02.9", "Acute pharyngitis, unspecified", is_primary=True))
    visit.add_attachment(Attachment("referral_letter.pdf", b"%PDF-1.4 scanned referral",
                                    description="Scanned referral from employer clinic"))

    lab_order = dr_brown.order_lab_test(visit, lab.get_test("FBC"), lab,
                                        notes="Check white cell count")
    prescription = dr_brown.prescribe(visit, [
        PrescriptionItem(pharmacy.get_medication("Amoxicillin"), "500 mg",
                         "1 capsule three times daily for 7 days", 21, 7),
        PrescriptionItem(pharmacy.get_medication("Paracetamol"), "1 g",
                         "2 tablets every 6 hours as needed for fever", 20, 5),
    ])
    prescription.send_to_pharmacy(pharmacy)
    print(key_values([("Visit", visit.visit_id), ("Vitals", visit.vitals),
                      ("Diagnosis", visit.diagnoses[0]),
                      ("Lab order", lab_order), ("Prescription", prescription)]))

    # The prescriber cannot give Aliyah the drug she is allergic to
    try:
        dr_brown.prescribe(visit, [PrescriptionItem(
            pharmacy.get_medication("Penicillin V"), "250 mg", "four times daily", 28)])
    except ValidationError as error:
        print(f"\n  Allergy check blocked prescription: {error}")

    # ================================================================ #
    # 5. Lab results and dispensing
    # ================================================================ #
    print(heading("Laboratory results and pharmacy"))
    technician = next(s for s in lab.staff if isinstance(s, LabTechnician))
    technician.collect_sample(lab_order)
    print(f"  Sample collected  -> {lab_order.status.value}")
    technician.upload_result(lab_order, aliyah, visit.visit_id,
                             {"WBC": "13.2 x10^9/L", "Neutrophils": "82 %", "Hb": "13.1 g/dL"},
                             "Neutrophilic leucocytosis consistent with bacterial infection",
                             is_abnormal=True)
    print(f"  Result uploaded   -> {lab_order.status.value}")
    print(f"  Result            : {lab_order.result}")

    pharmacist = next(s for s in pharmacy.staff if isinstance(s, Pharmacist))
    pharmacist.dispense(prescription)
    print(f"  Prescription      -> {prescription.status.value} by {pharmacist.full_name}")

    visit.close()
    print(f"  Appointment       -> {appointment.status.value}")

    # ================================================================ #
    # 6. Billing and payment
    # ================================================================ #
    print(heading("Billing and payment"))
    invoice = clinic.billing.generate_invoice(visit, other_services=[
        InvoiceLineItem("Nebuliser use", ChargeCategory.OTHER, 1, 12)])
    print(table(["Description", "Category", "Qty", "Unit", "Total"],
                [[i.description, i.category.value, i.quantity,
                  format_money(i.unit_price), format_money(i.total)]
                 for i in invoice.items], align_right=[2, 3, 4]))
    print()
    print(key_values([("Invoice", invoice.invoice_id),
                      ("Total charges", format_money(invoice.total)),
                      ("Insurance", invoice.policy),
                      ("Co-pay", format_money(invoice.co_pay)),
                      ("Insurer pays", format_money(invoice.insurer_share)),
                      ("Patient pays", format_money(invoice.patient_share)),
                      ("Status", invoice.status.value)]))

    claim = InsuranceClaimPayment(invoice.insurer_share, insurance).process(invoice)
    half = to_money(invoice.patient_share / 2)       # split between wallet and card
    wallet = MobileWalletPayment(half, "Lynk", "+1 876 555 0200").process(invoice)
    card = CardPayment(invoice.patient_outstanding, "4111 1111 1111 1111",
                       "Aliyah Johnson", "12/29").process(invoice)
    print(subheading("Payments"))
    print(table(["Payment", "Method", "Amount", "Reference", "Status"],
                [[p.payment_id, p.method_name, format_money(p.amount), p.reference,
                  p.status.value] for p in (claim, wallet, card)], align_right=[2]))
    print(f"\n  Balance due: {format_money(invoice.balance_due)} -> "
          f"invoice {invoice.status.value}")
    print(f"  Insurance limit remaining: {format_money(insurance.remaining_limit)}")
```

```
    # ================================================================ #
    # 7. Access control and visit summary download
    # ================================================================ #
    print(heading("Access control and visit summary"))
    other = clinic.register_patient("Jordan", "Clarke", date(1990, 3, 2), "+1 876 555 0300",
                                    "jordan.clarke@example.com", None, "jclarke", "Another#Pass1")
    try:
        aliyah.medical_record.get_visit(visit.visit_id, requester=other)
    except AccessDeniedError as error:
        print(f"  Other patient denied : {error}")
    seen = aliyah.medical_record.get_visit(visit.visit_id, requester=dr_brown)
    print(f"  Doctor access granted: {seen.visit_id}")

    path = aliyah.download_visit_summary(visit.visit_id)
    print(f"  Summary downloaded to: {path}\n")
    print(path.read_text(encoding="utf-8"))


if __name__ == "__main__":
    main()
```

tests/test_clinic_system.py

Test cases (separate file)

```
"""
test_clinic_system.py
=====================
Test cases for the Royal Care Clinic Management System.

Each ``TestCase`` class covers one feature from the assignment brief:

    TestClinicInformation          - clinic profile and departments
    TestPatientAccountCreation     - (a) patient account creation
    TestSearchAvailableAppointments- (b) searching for free slots
    TestMakeAppointment            - (c) booking / rescheduling
    TestVisitRecordingAndEMR       - (d) visit, diagnosis, vitals, access control
    TestPrescriptionGeneration     - (e) prescriptions sent to the pharmacy
    TestLabOrderLifecycle          - (f) lab order ORDERED -> COMPLETED
    TestInvoiceAndPayment          - (g) invoice, insurance, payments
    TestAppointmentCancellation    - (h) cancelling appointments
    TestExampleScenarios           - two complete end-to-end examples

Run from the project root with:
    python -m unittest discover -s tests -v
"""

import os
import sys
import tempfile
import unittest
from datetime import date, datetime, time, timedelta
from decimal import Decimal

# Make the project root importable when the file is run directly
sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))

from royal_care import (AccessDeniedError, AppointmentStatus, CardPayment,  # noqa: E402
                        ChargeCategory, Diagnosis, EmergencyContact,
                        InsuranceClaimPayment, InsurancePolicy, InvalidStateError,
                        InvoiceStatus, LabOrderStatus, LabTechnician, MobileWalletPayment,
                        Nurse, PaymentError, PaymentStatus, Pharmacist,
                        PrescriptionItem, PrescriptionStatus, SlotUnavailableError,
                        ValidationError, Vitals)
from royal_care.clinic import Address, Department  # noqa: E402
from royal_care.sample_data import build_royal_care_clinic  # noqa: E402
from royal_care.utils import IdGenerator, next_weekday  # noqa: E402


class ClinicTestBase(unittest.TestCase):
    """Shared fixture: a fresh clinic, one insured patient and helper methods."""

    def setUp(self) -> None:
```

```
        IdGenerator.reset()                          # independent IDs per test
        self.clinic = build_royal_care_clinic()
        self.monday = next_weekday(date.today(), 0)
        self.dr_brown = self.clinic.doctors("General Practice")[0]
        self.dr_okafor = self.clinic.doctors("Cardiology")[0]
        self.lab = self.clinic.laboratory()
        self.pharmacy = self.clinic.pharmacy()
        self.nurse = next(s for s in self.clinic.get_department("General Practice").staff
                          if isinstance(s, Nurse))
        self.technician = next(s for s in self.lab.staff if isinstance(s, LabTechnician))
        self.pharmacist = next(s for s in self.pharmacy.staff if isinstance(s, Pharmacist))
        self.policy = InsurancePolicy("Sagicor Health", "SH-1001", 80, 20, 5000,
                                      date.today() + timedelta(days=365))
        self.aliyah = self.clinic.register_patient(
            "Aliyah", "Johnson", date(1996, 7, 19), "+1 876 555 0199",
            "aliyah@example.com", Address("27 Palm Ave", "Kingston", "Jamaica"),
            "aliyahj", "Secure#2026",
            EmergencyContact("Marcus Johnson", "Brother", "+1 876 555 0177"),
            self.policy, ["Penicillin V"], "Mild asthma")

    # ---- helpers ---- #
    def at(self, hour: int, minute: int = 0, day: date = None) -> datetime:
        return datetime.combine(day or self.monday, time(hour, minute))

    def book_and_start_visit(self):
        appt = self.clinic.scheduler.book(self.aliyah, self.dr_brown, self.at(10), 30,
                                          "Sore throat")
        return appt, self.dr_brown.start_visit(appt)


# ============================================================================ #
class TestClinicInformation(ClinicTestBase):
    """Clinic profile: add, update and retrieve information."""

    def test_profile_is_retrievable(self):
        info = self.clinic.get_info()
        self.assertEqual(info["name"], "Royal Care Clinic")
        self.assertIn("+1 876 555 0100", info["phone_numbers"])
        for name in ("General Practice", "Cardiology", "Paediatrics",
                     "Laboratory", "Radiology", "Pharmacy"):
            self.assertIn(name, info["departments"])

    def test_update_clinic_info(self):
        self.clinic.update_info(name="Royal Care Clinic - Kingston")
        self.assertEqual(self.clinic.name, "Royal Care Clinic - Kingston")
        with self.assertRaises(ValidationError):
            self.clinic.update_info(owner="Someone")

    def test_department_add_update_and_duplicates(self):
        self.clinic.add_department(Department("Dermatology", location="Wing D"))
        self.clinic.update_department("Dermatology", phone="ext. 400")
        self.assertEqual(self.clinic.get_department("dermatology").phone, "ext. 400")
        with self.assertRaises(ValidationError):
            self.clinic.add_department(Department("Cardiology"))

    def test_department_operating_hours(self):
        gp = self.clinic.get_department("General Practice")
        self.assertEqual(gp.operating_hours.describe()[5], "Saturday: 09:00-13:00")
        self.assertIsNone(gp.operating_hours.hours_for(6))     # closed Sunday


# ============================================================================ #
class TestPatientAccountCreation(ClinicTestBase):
    """(a) Create a patient and confirm the data is stored correctly."""

    def test_patient_details_are_stored(self):
        stored = self.clinic.get_patient(self.aliyah.person_id)
        self.assertIs(stored, self.aliyah)
        self.assertEqual(stored.full_name, "Aliyah Johnson")
        self.assertEqual(stored.date_of_birth, date(1996, 7, 19))
        self.assertEqual(stored.emergency_contact.relationship, "Brother")
        self.assertEqual(stored.insurance.policy_number, "SH-1001")
        self.assertTrue(stored.is_allergic_to("penicillin v"))
        self.assertEqual(stored.account.username, "aliyahj")
        self.assertTrue(stored.person_id.startswith("PAT-"))

    def test_login_and_password_hashing(self):
```

```
        self.assertIs(self.clinic.login("aliyahj", "Secure#2026"), self.aliyah)
        with self.assertRaises(ValidationError):
            self.clinic.login("aliyahj", "wrong-password")

    def test_duplicate_username_and_invalid_data_rejected(self):
        with self.assertRaises(ValidationError):
            self.clinic.register_patient("A", "B", date(2000, 1, 1), "1", "a@b.com",
                                         None, "AliyahJ", "Password#1")
        with self.assertRaises(ValidationError):
            self.clinic.register_patient("A", "B", date(2000, 1, 1), "1", "not-an-email",
                                         None, "newuser", "Password#1")
        with self.assertRaises(ValidationError):
            self.clinic.register_patient("A", "B", date(2000, 1, 1), "1", "a@b.com",
                                         None, "newuser", "short")

    def test_update_profile(self):
        self.aliyah.update_profile(phone="+1 876 000 0000", email="new@example.com")
        self.assertEqual(self.aliyah.phone, "+1 876 000 0000")
        self.assertEqual(self.aliyah.email, "new@example.com")
        with self.assertRaises(ValidationError):
            self.aliyah.update_profile(date_of_birth=date(2000, 1, 1))


# ======================================================================== #
class TestSearchAvailableAppointments(ClinicTestBase):
    """(b) Doctor schedules produce the correct list of free slots."""

    def test_lists_all_slots_from_working_hours(self):
        slots = self.clinic.scheduler.search_available_slots(self.monday, doctor=self.dr_brown)
        # Dr Brown works 09:00-12:00 -> six 30-minute slots
        self.assertEqual(len(slots), 6)
        self.assertEqual(slots[0].slot.start, self.at(9))
        self.assertEqual(slots[-1].slot.end, self.at(12))

    def test_booked_slot_is_excluded(self):
        self.clinic.scheduler.book(self.aliyah, self.dr_brown, self.at(10))
        starts = [s.slot.start for s in
                  self.clinic.scheduler.search_available_slots(self.monday, doctor=self.dr_brown)]
        self.assertNotIn(self.at(10), starts)
        self.assertEqual(len(starts), 5)

    def test_search_by_speciality_and_date(self):
        cardio = self.clinic.scheduler.search_available_slots(self.monday, speciality="cardiology")
        self.assertTrue(cardio)
        self.assertTrue(all(s.doctor is self.dr_okafor for s in cardio))
        all_slots = self.clinic.scheduler.search_available_slots(self.monday)
        self.assertEqual(len(all_slots), 6 + 6 + 12)        # Brown + Okafor + Patel
        saturday = self.monday + timedelta(days=5)
        self.assertEqual(self.clinic.scheduler.search_available_slots(saturday), [])

    def test_custom_duration(self):
        slots = self.clinic.scheduler.search_available_slots(
            self.monday, doctor=self.dr_brown, duration_minutes=60)
        self.assertEqual(len(slots), 3)


# ======================================================================== #
class TestMakeAppointment(ClinicTestBase):
    """(c) Book an appointment with a doctor and confirm its status."""

    def test_booking_confirms_status(self):
        appt = self.clinic.scheduler.book(self.aliyah, self.dr_brown, self.at(9, 30),
                                          30, "Check-up")
        self.assertEqual(appt.status, AppointmentStatus.BOOKED)
        self.assertIs(appt.doctor, self.dr_brown)
        self.assertEqual(appt.slot.duration_minutes, 30)
        self.assertEqual(self.aliyah.upcoming_appointments(self.clinic.scheduler), [appt])

    def test_double_booking_and_out_of_hours_rejected(self):
        self.clinic.scheduler.book(self.aliyah, self.dr_brown, self.at(10))
        other = self.clinic.register_patient("Jo", "Doe", date(1980, 1, 1), "1",
                                             "jo@example.com", None, "jodoe", "Password#1")
        with self.assertRaises(SlotUnavailableError):
            self.clinic.scheduler.book(other, self.dr_brown, self.at(10, 15))
        with self.assertRaises(SlotUnavailableError):
            self.clinic.scheduler.book(other, self.dr_brown, self.at(15))
        with self.assertRaises(SlotUnavailableError):       # patient already busy
```

```
            self.clinic.scheduler.book(self.aliyah, self.clinic.doctors("Paediatrics")[0],
                                       self.at(10))

    def test_reschedule(self):
        appt = self.clinic.scheduler.book(self.aliyah, self.dr_brown, self.at(9))
        self.clinic.scheduler.reschedule(appt.appointment_id, self.at(11))
        self.assertEqual(appt.status, AppointmentStatus.RESCHEDULED)
        self.assertEqual(appt.slot.start, self.at(11))
        self.assertEqual(len(appt.history), 2)
        # the old slot is free again
        starts = [s.slot.start for s in
                  self.clinic.scheduler.search_available_slots(self.monday, doctor=self.dr_brown)]
        self.assertIn(self.at(9), starts)


# ============================================================================ #
class TestVisitRecordingAndEMR(ClinicTestBase):
    """(d) Record a visit with diagnosis and vitals linked to the appointment."""

    def test_visit_links_to_appointment_and_record(self):
        appt, visit = self.book_and_start_visit()
        self.nurse.record_vitals(visit, Vitals(38.1, 120, 80, 92, 18, 64, 167))
        visit.add_diagnosis(Diagnosis("J02.9", "Acute pharyngitis", is_primary=True))
        self.assertIs(visit.appointment, appt)
        self.assertIs(appt.visit, visit)
        self.assertIs(visit.patient, self.aliyah)
        self.assertEqual(visit.vitals.temperature_c, 38.1)
        self.assertEqual(visit.vitals.bmi, 22.9)
        self.assertEqual(visit.diagnoses[0].code, "J02.9")
        self.assertIs(self.aliyah.medical_record.get_visit(visit.visit_id, self.dr_brown), visit)

    def test_closing_visit_completes_appointment_and_locks_it(self):
        appt, visit = self.book_and_start_visit()
        visit.close()
        self.assertEqual(appt.status, AppointmentStatus.COMPLETED)
        with self.assertRaises(InvalidStateError):
            visit.add_note("late note", self.dr_brown)

    def test_validation_rules(self):
        _, visit = self.book_and_start_visit()
        visit.add_diagnosis(Diagnosis("J02.9", "Pharyngitis", is_primary=True))
        with self.assertRaises(ValidationError):
            visit.add_diagnosis(Diagnosis("R50.9", "Fever", is_primary=True))
        with self.assertRaises(ValidationError):
            Vitals(55, 120, 80, 80, 16, 70, 170)           # impossible temperature
        with self.assertRaises(InvalidStateError):
            self.dr_okafor.start_visit(visit.appointment)   # not his appointment

    def test_access_control_and_summary_download(self):
        _, visit = self.book_and_start_visit()
        visit.add_diagnosis(Diagnosis("J02.9", "Acute pharyngitis"))
        stranger = self.clinic.register_patient("Sam", "Lee", date(1985, 5, 5), "1",
                                                "sam@example.com", None, "samlee", "Password#1")
        with self.assertRaises(AccessDeniedError):
            self.aliyah.medical_record.get_visit(visit.visit_id, requester=stranger)
        self.assertEqual(len(self.aliyah.medical_record.visits(self.nurse)), 1)
        with tempfile.TemporaryDirectory() as folder:
            path = self.aliyah.download_visit_summary(visit.visit_id, folder)
            text = path.read_text(encoding="utf-8")
        self.assertIn("Acute pharyngitis", text)
        self.assertIn(visit.visit_id, text)


# ============================================================================ #
class TestPrescriptionGeneration(ClinicTestBase):
    """(e) Doctor creates a prescription; it is marked as sent to the pharmacy."""

    def test_prescription_sent_to_pharmacy(self):
        _, visit = self.book_and_start_visit()
        rx = self.dr_brown.prescribe(visit, [PrescriptionItem(
            self.pharmacy.get_medication("Amoxicillin"), "500 mg",
            "1 capsule three times daily", 21, 7)])
        self.assertEqual(rx.status, PrescriptionStatus.DRAFT)
        rx.send_to_pharmacy(self.pharmacy)
        self.assertEqual(rx.status, PrescriptionStatus.SENT_TO_PHARMACY)
        self.assertIn(rx, self.pharmacy.received_prescriptions)
        self.assertEqual(rx.items[0].drug_name, "Amoxicillin")
```

```
        self.assertEqual(rx.total_cost, Decimal("16.80"))
        self.pharmacist.dispense(rx)
        self.assertEqual(rx.status, PrescriptionStatus.DISPENSED)

    def test_allergy_and_state_rules(self):
        _, visit = self.book_and_start_visit()
        with self.assertRaises(ValidationError):
            self.dr_brown.prescribe(visit, [PrescriptionItem(
                self.pharmacy.get_medication("Penicillin V"), "250 mg", "four times daily", 28)])
        rx = self.dr_brown.prescribe(visit, [PrescriptionItem(
            self.pharmacy.get_medication("Paracetamol"), "1 g", "every 6 hours", 20)])
        rx.send_to_pharmacy(self.pharmacy)
        with self.assertRaises(InvalidStateError):
            rx.add_item(PrescriptionItem(self.pharmacy.get_medication("Ibuprofen"),
                                         "400 mg", "twice daily", 10))
        with self.assertRaises(InvalidStateError):
            rx.send_to_pharmacy(self.pharmacy)


# ============================================================================ #
class TestLabOrderLifecycle(ClinicTestBase):
    """(f) Doctor orders a lab test, result received, status COMPLETED."""

    def test_full_lifecycle(self):
        _, visit = self.book_and_start_visit()
        order = self.dr_brown.order_lab_test(visit, self.lab.get_test("FBC"), self.lab)
        self.assertEqual(order.status, LabOrderStatus.ORDERED)
        self.assertIn(order, self.lab.pending_orders())
        self.technician.collect_sample(order)
        self.assertEqual(order.status, LabOrderStatus.SAMPLE_COLLECTED)
        self.technician.upload_result(order, self.aliyah, visit.visit_id,
                                      {"WBC": "13.2"}, "Raised WBC", is_abnormal=True)
        self.assertEqual(order.status, LabOrderStatus.COMPLETED)
        self.assertIs(order.patient, self.aliyah)
        self.assertIs(order.visit, visit)
        self.assertTrue(order.result.is_abnormal)
        self.assertNotIn(order, self.lab.pending_orders())

    def test_result_must_match_patient_and_visit(self):
        _, visit = self.book_and_start_visit()
        order = self.dr_brown.order_lab_test(visit, self.lab.get_test("FBC"), self.lab)
        other = self.clinic.register_patient("Sam", "Lee", date(1985, 5, 5), "1",
                                             "sam@example.com", None, "samlee", "Password#1")
        with self.assertRaises(ValidationError):
            self.technician.upload_result(order, other, visit.visit_id, {"WBC": "5"}, "ok")
        with self.assertRaises(ValidationError):
            self.technician.upload_result(order, self.aliyah, "VIS-9999", {"WBC": "5"}, "ok")
        self.assertEqual(order.status, LabOrderStatus.ORDERED)


# ============================================================================ #
class TestInvoiceAndPayment(ClinicTestBase):
    """(g) Generate invoice, apply insurance and process payment."""

    def _billable_visit(self):
        _, visit = self.book_and_start_visit()
        self.dr_brown.order_lab_test(visit, self.lab.get_test("FBC"), self.lab)
        self.dr_brown.prescribe(visit, [PrescriptionItem(
            self.pharmacy.get_medication("Amoxicillin"), "500 mg", "tds", 21)])
        visit.close()
        return visit

    def test_invoice_lines_and_insurance(self):
        invoice = self.clinic.billing.generate_invoice(self._billable_visit())
        # 80.00 consultation + 25.00 FBC + 21 x 0.80 amoxicillin = 121.80
        self.assertEqual(invoice.total, Decimal("121.80"))
        self.assertEqual(invoice.subtotal(ChargeCategory.CONSULTATION), Decimal("80.00"))
        self.assertEqual(invoice.subtotal(ChargeCategory.LAB), Decimal("25.00"))
        self.assertEqual(invoice.subtotal(ChargeCategory.MEDICATION), Decimal("16.80"))
        # (121.80 - 20 co-pay) x 80% = 81.44
        self.assertEqual(invoice.co_pay, Decimal("20.00"))
        self.assertEqual(invoice.insurer_share, Decimal("81.44"))
        self.assertEqual(invoice.patient_share, Decimal("40.36"))
        self.assertEqual(invoice.status, InvoiceStatus.UNPAID)

    def test_payment_process(self):
        invoice = self.clinic.billing.generate_invoice(self._billable_visit())
```

```
        InsuranceClaimPayment(invoice.insurer_share, self.policy).process(invoice)
        self.assertEqual(invoice.status, InvoiceStatus.PARTIALLY_PAID)
        card = CardPayment(invoice.patient_outstanding, "4111 1111 1111 1111",
                           "Aliyah Johnson", "12/30").process(invoice)
        self.assertEqual(card.status, PaymentStatus.SUCCESSFUL)
        self.assertEqual(invoice.balance_due, Decimal("0.00"))
        self.assertEqual(invoice.status, InvoiceStatus.PAID)
        self.assertEqual(self.policy.remaining_limit, Decimal("4918.56"))

    def test_rejected_payments(self):
        invoice = self.clinic.billing.generate_invoice(self._billable_visit())
        with self.assertRaises(PaymentError):              # fails Luhn check
            CardPayment(10, "4111 1111 1111 1112", "A J", "12/30").process(invoice)
        with self.assertRaises(PaymentError):              # more than the patient owes
            MobileWalletPayment(500, "Lynk", "+18765550199").process(invoice)
        with self.assertRaises(InvalidStateError):         # visit already invoiced
            self.clinic.billing.generate_invoice(invoice.visit)
        self.assertEqual(invoice.status, InvoiceStatus.UNPAID)

    def test_coverage_limits_and_expired_policy(self):
        self.policy.annual_limit = Decimal("50.00")
        invoice = self.clinic.billing.generate_invoice(self._billable_visit())
        self.assertEqual(invoice.insurer_share, Decimal("50.00"))   # capped at limit
        expired = InsurancePolicy("Old Insurer", "X-1", 90, 0, 1000, date(2020, 1, 1))
        with self.assertRaises(ValidationError):
            invoice.apply_insurance(expired)


# ============================================================================ #
class TestAppointmentCancellation(ClinicTestBase):
    """(h) Cancel an appointment."""

    def test_cancel_frees_slot(self):
        appt = self.clinic.scheduler.book(self.aliyah, self.dr_brown, self.at(10))
        self.clinic.scheduler.cancel(appt.appointment_id, "Feeling better")
        self.assertEqual(appt.status, AppointmentStatus.CANCELLED)
        self.assertEqual(appt.cancellation_reason, "Feeling better")
        self.assertEqual(self.aliyah.upcoming_appointments(self.clinic.scheduler), [])
        starts = [s.slot.start for s in
                  self.clinic.scheduler.search_available_slots(self.monday, doctor=self.dr_brown)]
        self.assertIn(self.at(10), starts)

    def test_cancelled_appointment_cannot_be_used(self):
        appt = self.clinic.scheduler.book(self.aliyah, self.dr_brown, self.at(10))
        self.clinic.scheduler.cancel(appt.appointment_id)
        with self.assertRaises(InvalidStateError):
            self.clinic.scheduler.cancel(appt.appointment_id)
        with self.assertRaises(InvalidStateError):
            self.clinic.scheduler.reschedule(appt.appointment_id, self.at(11))
        with self.assertRaises(InvalidStateError):
            self.dr_brown.start_visit(appt)


# ============================================================================ #
class TestExampleScenarios(ClinicTestBase):
    """Two complete end-to-end examples."""

    def test_example_1_aliyah_and_dr_brown(self):
        """Aliyah books 30 min next Monday with Dr. Brown (GP); full visit to payment."""
        # Patient and doctor exist
        self.assertEqual(self.dr_brown.full_name, "Dr. Michael Brown")
        self.assertEqual(self.dr_brown.speciality, "General Practice")

        # Booking a 30-minute appointment next Monday
        slot = self.clinic.scheduler.search_available_slots(
            self.monday, doctor=self.dr_brown)[0]
        appt = self.clinic.scheduler.book(self.aliyah, self.dr_brown, slot.slot.start, 30,
                                          "Sore throat and fever")
        self.assertEqual(appt.slot.start.weekday(), 0)
        self.assertEqual(appt.slot.duration_minutes, 30)

        # Visit: vitals, diagnosis, lab order, prescription
        visit = self.dr_brown.start_visit(appt)
        self.nurse.record_vitals(visit, Vitals(38.4, 122, 78, 96, 18, 64.5, 167))
        visit.add_diagnosis(Diagnosis("J02.9", "Acute pharyngitis", is_primary=True))
        order = self.dr_brown.order_lab_test(visit, self.lab.get_test("STREP"), self.lab)
        rx = self.dr_brown.prescribe(visit, [
```

```
            PrescriptionItem(self.pharmacy.get_medication("Amoxicillin"), "500 mg",
                             "1 capsule three times daily for 7 days", 21, 7),
            PrescriptionItem(self.pharmacy.get_medication("Paracetamol"), "1 g",
                             "2 tablets every 6 hours as needed", 20, 5)])
        rx.send_to_pharmacy(self.pharmacy)

        # Lab results uploaded
        self.technician.upload_result(order, self.aliyah, visit.visit_id,
                                      {"Strep A antigen": "Positive"},
                                      "Group A streptococcal pharyngitis", True)
        visit.close()

        # Invoice and payment
        invoice = self.clinic.billing.generate_invoice(visit)
        InsuranceClaimPayment(invoice.insurer_share, self.policy).process(invoice)
        CardPayment(invoice.patient_outstanding, "5555 5555 5555 4444",
                    "Aliyah Johnson", "08/29").process(invoice)

        # Assertions: visit data, prescription and billing are accurate
        self.assertEqual(appt.status, AppointmentStatus.COMPLETED)
        self.assertEqual(visit.diagnoses[0].description, "Acute pharyngitis")
        self.assertEqual(visit.vitals.blood_pressure, "122/78 mmHg")
        self.assertEqual(order.status, LabOrderStatus.COMPLETED)
        self.assertEqual(rx.status, PrescriptionStatus.SENT_TO_PHARMACY)
        self.assertEqual([i.drug_name for i in rx.items], ["Amoxicillin", "Paracetamol"])
        # 80 + 18 + 16.80 + 2.00 = 116.80; insurer (116.80 - 20) x 0.8 = 77.44
        self.assertEqual(invoice.total, Decimal("116.80"))
        self.assertEqual(invoice.insurer_share, Decimal("77.44"))
        self.assertEqual(invoice.patient_share, Decimal("39.36"))
        self.assertEqual(invoice.status, InvoiceStatus.PAID)

    def test_example_2_uninsured_cardiology_patient(self):
        """Marcus (no insurance) is rescheduled, seen by Dr. Okafor and pays by wallet."""
        marcus = self.clinic.register_patient(
            "Marcus", "Grant", date(1962, 2, 14), "+1 876 555 0400",
            "marcus.grant@example.com", None, "mgrant", "Heart#Health1",
            medical_history_summary="Hypertension, family history of heart disease")
        wednesday = self.monday + timedelta(days=2)
        appt = self.clinic.scheduler.book(marcus, self.dr_okafor,
                                          self.at(13, 0, wednesday), 30, "Chest tightness")
        self.clinic.scheduler.reschedule(appt.appointment_id, self.at(14, 30, wednesday))
        self.assertEqual(appt.status, AppointmentStatus.RESCHEDULED)

        visit = self.dr_okafor.start_visit(appt)
        self.nurse.record_vitals(visit, Vitals(36.8, 148, 94, 82, 16, 92, 178))
        visit.add_diagnosis(Diagnosis("I10", "Essential hypertension", is_primary=True))
        visit.add_diagnosis(Diagnosis("E78.5", "Hyperlipidaemia"))
        order = self.dr_okafor.order_lab_test(visit, self.lab.get_test("LIPID"), self.lab)
        rx = self.dr_okafor.prescribe(visit, [PrescriptionItem(
            self.pharmacy.get_medication("Atorvastatin"), "20 mg",
            "1 tablet at night", 30, 30)])
        rx.send_to_pharmacy(self.pharmacy)
        self.pharmacist.dispense(rx)
        self.technician.upload_result(order, marcus, visit.visit_id,
                                      {"LDL": "4.6 mmol/L", "HDL": "1.0 mmol/L"},
                                      "Raised LDL cholesterol", True)
        visit.close()

        invoice = self.clinic.billing.generate_invoice(visit)
        # 150 consultation + 40 lipid panel + 30 x 0.50 atorvastatin = 205.00
        self.assertEqual(invoice.total, Decimal("205.00"))
        self.assertEqual(invoice.insurer_share, Decimal("0.00"))
        self.assertEqual(invoice.patient_share, Decimal("205.00"))
        MobileWalletPayment(205, "Lynk", "+1 876 555 0400").process(invoice)

        self.assertEqual(invoice.status, InvoiceStatus.PAID)
        self.assertEqual(rx.status, PrescriptionStatus.DISPENSED)
        self.assertEqual(order.result.values["LDL"], "4.6 mmol/L")
        self.assertEqual(len(marcus.medical_record.visits(marcus)), 1)
        self.assertEqual(visit.vitals.bmi, 29.0)


if __name__ == "__main__":
    unittest.main(verbosity=2)
```

Program output (python main.py)

```
================================================================================
                     ROYAL CARE CLINIC MANAGEMENT SYSTEM
================================================================================

--- Clinic profile -------------------------------------------------------------
  Name    : Royal Care Clinic
  Address : 14 Harbour View Road, Kingston, KN10, Jamaica
  Phone   : +1 876 555 0100, +1 876 555 0101
  Email   : info@royalcareclinic.example
  Website : www.royalcareclinic.example

--- Departments ----------------------------------------------------------------
  Department       | Type       | Location             | Mon hours   | Sat hours   | Staff
  -----------------+------------+----------------------+-------------+-------------+------
  General Practice | Department | Ground floor, Wing A | 08:00-18:00 | 09:00-13:00 | 2
  Cardiology       | Department | First floor, Wing B  | 08:00-17:00 | Closed      | 1
  Paediatrics      | Department | Ground floor, Wing C | 08:00-17:00 | Closed      | 1
  Radiology        | Department | Basement             | 08:00-17:00 | Closed      | 0
  Laboratory       | Laboratory | Basement             | 08:00-17:00 | Closed      | 1
  Pharmacy         | Pharmacy   | Ground floor, Lobby  | 08:00-17:00 | Closed      | 1

  Updated Radiology phone -> ext. 111

--- Doctors --------------------------------------------------------------------
  ID       | Name              | Speciality       | Qualifications   |     Fee
  ---------+-------------------+------------------+------------------+--------
  DOC-0001 | Dr. Michael Brown | General Practice | MBBS, DFM        |  $80.00
  DOC-0002 | Dr. Ngozi Okafor  | Cardiology       | MBBS, MRCP, FACC | $150.00
  DOC-0003 | Dr. Ravi Patel    | Paediatrics      | MBBS, DCH        |  $95.00

================================================================================
                             PATIENT REGISTRATION
================================================================================
  Patient ID        : PAT-0001
  Name              : Aliyah Johnson
  Age               : 30
  Email             : aliyah.johnson@example.com
  Emergency contact : Marcus Johnson (Brother) - +1 876 555 0177
  Insurance         : Sagicor Health #SH-778812 - 80% cover, co-pay $20.00
  Allergies         : Penicillin V
  History           : Mild asthma (childhood); no surgeries.
  Login check       : OK

  Profile updated: new phone +1 876 555 0200

================================================================================
                            APPOINTMENT SCHEDULING
================================================================================

--- Available GP slots on Monday 02 November 2026 ----------------------------
  # | Doctor            | Slot
  --+-------------------+---------------------------
  1 | Dr. Michael Brown | Mon 02 Nov 2026 09:00-09:30
  2 | Dr. Michael Brown | Mon 02 Nov 2026 09:30-10:00
  3 | Dr. Michael Brown | Mon 02 Nov 2026 10:00-10:30
  4 | Dr. Michael Brown | Mon 02 Nov 2026 10:30-11:00
  5 | Dr. Michael Brown | Mon 02 Nov 2026 11:00-11:30
  6 | Dr. Michael Brown | Mon 02 Nov 2026 11:30-12:00

  Booked   : APT-0001: Aliyah Johnson with Dr. Michael Brown on Mon 02 Nov 2026 09:30-10:00 [BOOKED]
  Moved    : APT-0001: Aliyah Johnson with Dr. Michael Brown on Mon 02 Nov 2026 10:00-10:30 [RESCHEDULED]
  Cancelled: APT-0002: Aliyah Johnson with Dr. Ngozi Okafor on Wed 04 Nov 2026 14:00-14:30 [CANCELLED]

--- Aliyah's upcoming appointments (patient portal) --------------------------
  APT-0001: Aliyah Johnson with Dr. Michael Brown on Mon 02 Nov 2026 10:00-10:30 [RESCHEDULED]

================================================================================
                     VISIT AND ELECTRONIC MEDICAL RECORD
================================================================================
  Visit        : VIS-0001
  Vitals       : Temp 38.4 °C | BP 122/78 mmHg | Pulse 96 bpm | RR 18/min | SpO2 97% | BMI 23.1
  Diagnosis    : J02.9 - Acute pharyngitis, unspecified (primary)
  Lab order    : LBO-0001: Full Blood Count for Aliyah Johnson [ORDERED]
  Prescription : RX-0001 for Aliyah Johnson by Dr. Michael Brown [SENT_TO_PHARMACY]

  Allergy check blocked prescription: Aliyah Johnson is allergic to Penicillin V.

================================================================================
                       LABORATORY RESULTS AND PHARMACY
================================================================================
  Sample collected  -> SAMPLE_COLLECTED
```

```
  Result uploaded   -> COMPLETED
  Result            : WBC: 13.2 x10^9/L, Neutrophils: 82 %, Hb: 13.1 g/dL -> Neutrophilic leucocytosis consistent
with bacterial infection (ABNORMAL)
  Prescription      -> DISPENSED by Sophia Campbell
  Appointment       -> COMPLETED

================================================================================
                            BILLING AND PAYMENT
================================================================================
  Description                                          | Category      | Qty |   Unit |  Total
  -----------------------------------------------------+---------------+-----+--------+-------
  Consultation - Dr. Michael Brown (General Practice) | Consultation  |   1 | $80.00 | $80.00
  Lab test - Full Blood Count                          | Laboratory    |   1 | $25.00 | $25.00
  Medication - Amoxicillin 500 mg capsule              | Medication    |  21 |  $0.80 | $16.80
  Medication - Paracetamol 500 mg tablet               | Medication    |  20 |  $0.10 |  $2.00
  Nebuliser use                                        | Other Service |   1 | $12.00 | $12.00

  Invoice       : INV-0001
  Total charges : $135.80
  Insurance     : Sagicor Health #SH-778812 - 80% cover, co-pay $20.00
  Co-pay        : $20.00
  Insurer pays  : $92.64
  Patient pays  : $43.16
  Status        : UNPAID

--- Payments ---------------------------------------------------------------
  Payment  | Method                           | Amount | Reference               | Status
  ---------+----------------------------------+--------+-------------------------+-----------
  PAY-0001 | Insurance Claim (Sagicor Health) | $92.64 | CLM-0001                | SUCCESSFUL
  PAY-0002 | Mobile Wallet (Lynk)             | $21.58 | WALLET-LYNK-PAY-0002    | SUCCESSFUL
  PAY-0003 | Card                             | $21.58 | CARD-AUTH-1111-PAY-0003 | SUCCESSFUL

  Balance due: $0.00 -> invoice PAID
  Insurance limit remaining: $4,907.36

================================================================================
                       ACCESS CONTROL AND VISIT SUMMARY
================================================================================
  Other patient denied : Jordan Clarke is not allowed to view this medical record.
  Doctor access granted: VIS-0001
  Summary downloaded to: output/visit_summary_VIS-0001.txt

ROYAL CARE CLINIC - VISIT SUMMARY
==================================================
Visit ID     : VIS-0001
Appointment  : APT-0001 (Mon 02 Nov 2026 10:00-10:30)
Patient      : Aliyah Johnson (PAT-0001)
Doctor       : Dr. Michael Brown - General Practice
Reason       : Sore throat and fever for 3 days

Vitals       : Temp 38.4 °C | BP 122/78 mmHg | Pulse 96 bpm | RR 18/min | SpO2 97% | BMI 23.1
Diagnoses    :
  - J02.9 - Acute pharyngitis, unspecified (primary)
Prescriptions:
  - Amoxicillin 500 mg capsule - 500 mg, 1 capsule three times daily for 7 days (qty 21)
  - Paracetamol 500 mg tablet - 1 g, 2 tablets every 6 hours as needed for fever (qty 20)
Lab orders   :
  - Full Blood Count: COMPLETED -> Neutrophilic leucocytosis consistent with bacterial infection
Notes        :
  - Erythematous pharynx with tonsillar exudate; no cough.
Attachments  :
  - referral_letter.pdf (application/pdf, 25 bytes)
```

Test results (python -m unittest discover -s tests -v)

The tests cover features a–h from the brief, plus clinic information. They also include two example scenarios: (1) Aliyah with Dr. Brown, and (2) uninsured patient Marcus with cardiologist Dr. Okafor, who is rescheduled and pays by mobile wallet.

Brief item	Test class	What is verified
a	TestPatientAccountCreation	Details stored, password login, duplicate/invalid data rejected, profile update

b	TestSearchAvailableAppointments	Slots generated from schedules, booked slots excluded, filter by doctor/speciality/date/duration
c	TestMakeAppointment	Status BOOKED, double-booking and out-of-hours rejected, rescheduling
d	TestVisitRecordingAndEMR	Visit linked to appointment, vitals and diagnosis, closing, access control, summary download
e	TestPrescriptionGeneration	SENT_TO_PHARMACY status, allergy block, no edits after sending, dispensing
f	TestLabOrderLifecycle	ORDERED → SAMPLE_COLLECTED → COMPLETED; wrong patient/visit rejected
g	TestInvoiceAndPayment	Line totals, co-pay and insurer share, payments to PAID, invalid card, overpayment, limit cap, expired policy
h	TestAppointmentCancellation	CANCELLED status, slot freed, no reuse of cancelled appointment
Examples	TestExampleScenarios	Two complete end-to-end scenarios with assertions on visit, prescription and billing
Extra	TestClinicInformation	Profile retrieval, update, departments and operating hours

```
test_cancel_frees_slot (test_clinic_system.TestAppointmentCancellation.test_cancel_frees_slot) ... ok
test_cancelled_appointment_cannot_be_used
(test_clinic_system.TestAppointmentCancellation.test_cancelled_appointment_cannot_be_used) ... ok
test_department_add_update_and_duplicates
(test_clinic_system.TestClinicInformation.test_department_add_update_and_duplicates) ... ok
test_department_operating_hours (test_clinic_system.TestClinicInformation.test_department_operating_hours) ... ok
test_profile_is_retrievable (test_clinic_system.TestClinicInformation.test_profile_is_retrievable) ... ok
test_update_clinic_info (test_clinic_system.TestClinicInformation.test_update_clinic_info) ... ok
test_example_1_aliyah_and_dr_brown (test_clinic_system.TestExampleScenarios.test_example_1_aliyah_and_dr_brown)
Aliyah books 30 min next Monday with Dr. Brown (GP); full visit to payment. ... ok
test_example_2_uninsured_cardiology_patient
(test_clinic_system.TestExampleScenarios.test_example_2_uninsured_cardiology_patient)
Marcus (no insurance) is rescheduled, seen by Dr. Okafor and pays by wallet. ... ok
test_coverage_limits_and_expired_policy
(test_clinic_system.TestInvoiceAndPayment.test_coverage_limits_and_expired_policy) ... ok
test_invoice_lines_and_insurance (test_clinic_system.TestInvoiceAndPayment.test_invoice_lines_and_insurance) ... ok
test_payment_process (test_clinic_system.TestInvoiceAndPayment.test_payment_process) ... ok
test_rejected_payments (test_clinic_system.TestInvoiceAndPayment.test_rejected_payments) ... ok
test_full_lifecycle (test_clinic_system.TestLabOrderLifecycle.test_full_lifecycle) ... ok
test_result_must_match_patient_and_visit
(test_clinic_system.TestLabOrderLifecycle.test_result_must_match_patient_and_visit) ... ok
test_booking_confirms_status (test_clinic_system.TestMakeAppointment.test_booking_confirms_status) ... ok
test_double_booking_and_out_of_hours_rejected
(test_clinic_system.TestMakeAppointment.test_double_booking_and_out_of_hours_rejected) ... ok
test_reschedule (test_clinic_system.TestMakeAppointment.test_reschedule) ... ok
test_duplicate_username_and_invalid_data_rejected
(test_clinic_system.TestPatientAccountCreation.test_duplicate_username_and_invalid_data_rejected) ... ok
test_login_and_password_hashing (test_clinic_system.TestPatientAccountCreation.test_login_and_password_hashing) ...
ok
test_patient_details_are_stored (test_clinic_system.TestPatientAccountCreation.test_patient_details_are_stored) ...
ok
test_update_profile (test_clinic_system.TestPatientAccountCreation.test_update_profile) ... ok
test_allergy_and_state_rules (test_clinic_system.TestPrescriptionGeneration.test_allergy_and_state_rules) ... ok
test_prescription_sent_to_pharmacy
(test_clinic_system.TestPrescriptionGeneration.test_prescription_sent_to_pharmacy) ... ok
test_booked_slot_is_excluded (test_clinic_system.TestSearchAvailableAppointments.test_booked_slot_is_excluded) ...
ok
test_custom_duration (test_clinic_system.TestSearchAvailableAppointments.test_custom_duration) ... ok
test_lists_all_slots_from_working_hours
(test_clinic_system.TestSearchAvailableAppointments.test_lists_all_slots_from_working_hours) ... ok
test_search_by_speciality_and_date
(test_clinic_system.TestSearchAvailableAppointments.test_search_by_speciality_and_date) ... ok
test_access_control_and_summary_download
(test_clinic_system.TestVisitRecordingAndEMR.test_access_control_and_summary_download) ... ok
test_closing_visit_completes_appointment_and_locks_it
(test_clinic_system.TestVisitRecordingAndEMR.test_closing_visit_completes_appointment_and_locks_it) ... ok
test_validation_rules (test_clinic_system.TestVisitRecordingAndEMR.test_validation_rules) ... ok
test_visit_links_to_appointment_and_record
(test_clinic_system.TestVisitRecordingAndEMR.test_visit_links_to_appointment_and_record) ... ok

----------------------------------------------------------------------
Ran 31 tests in 2.042s
```

```
OK
```

c. GitHub Repository

Repository link (public): https://github.com/<your-username>/royal-care-clinic

The repository contains the `royal_care` package, `main.py`, the `tests` folder, the UML source and images in `docs/`, and a README with run instructions. The commit history below shows the system being built module by module.

Commit	Date	Message
`0d8d8e9`	28 Oct 2026 13:56	Set up project structure, enums, exceptions and utilities
`f06d1d9`	28 Oct 2026 14:00	Add clinic profile, departments, laboratory/pharmacy and patient accounts
`d9d7d0c`	28 Oct 2026 14:00	Add Person hierarchy: Patient, Staff, Doctor, Nurse, LabTechnician, Pharmacist
`08e7904`	28 Oct 2026 14:00	Add appointment scheduling: slot search, booking, rescheduling, cancellation
`c3779d9`	28 Oct 2026 14:00	Add electronic medical records with visits, vitals, diagnoses and access control
`66b1570`	28 Oct 2026 14:00	Add prescriptions and laboratory order lifecycle
`b340de5`	28 Oct 2026 14:00	Add billing, insurance co-pay calculation and payment methods
`4f2bac3`	28 Oct 2026 14:01	Add sample clinic data and end-to-end demonstration script
`ce787cb`	28 Oct 2026 14:03	Add test cases for features a-h and two end-to-end example scenarios
`36973de`	28 Oct 2026 14:05	Add UML class diagram (PlantUML source, PNG and SVG) and expand README

d. Summary of Learnings

[Draft. Edit this so it reflects your own experience of the assignment.]

Choosing the right relationship matters. Deciding between composition, aggregation and plain association made me think about object lifetimes. A Visit cannot exist without a MedicalRecord, but a Staff member clearly outlives their Department. The UML symbols turned into real decisions in the code: composed parts are created inside the owner's constructor, while aggregated parts are passed in.

Inheritance and polymorphism reduce duplication. Putting shared personal data in the abstract Person class, and staff data in Staff, meant each role subclass only needed its own behaviour. The Payment hierarchy showed the template-method pattern: the validation and recording steps are written once in Payment.process(), and each subclass supplies only its authorisation step. Invoice never needs to know which kind of payment it has received.

Encapsulation protects data integrity. Private attributes with read-only properties, plus state-changing methods that check the current status, make illegal states impossible. A cancelled appointment cannot be rescheduled. A prescription cannot be edited after it is sent. A lab result cannot be filed against the wrong patient or visit. This matters for medical data.

Modularity needs planning. Splitting the system into one module per area kept each file focused. It also caused circular-import problems (people ↔ emr ↔ scheduling), which I solved with type-only imports (TYPE_CHECKING) and by keeping the dependency direction clear.

Testing drives confidence. Writing a test class for each feature, plus two end-to-end scenarios, made me state the expected results precisely, for example the co-pay arithmetic. It also exposed edge cases such as double booking, expired policies and overpayment. Using Decimal instead of float avoided rounding errors in the billing tests.

Version control shows progress. Committing each module separately produced a readable history of how the system was built.